
CYBERSECURITY FOR PROFESSIONALS BOOTCAMP

HANDS-ON LAB

Threat Hunting and Detection Engineering

By: Dominic Yeakel

Week 6, Days 4–5 • Browser-Based Lab • Individual Lab

1. Before You Start

This lab continues directly from the forensic investigation you completed on Day 3. It runs entirely in your web browser. There is nothing to install, no virtual machine to connect to, no account to create, and no password to request. Everything you need is printed inside this document. The websites confirm what you already have; they do not hold anything you cannot see here.

1.1 What you need

Item	Detail
A web browser	Chrome, Edge, Firefox or Safari. Any current version is fine.
A spreadsheet	Excel, Google Sheets or LibreOffice Calc. Most of Day 4 happens here.
A plain text editor	Notepad, TextEdit, VS Code, or anything that saves .txt files. You will write detection rules on Day 5.
A document editor	Word, Google Docs or anything that produces a PDF for your reports.
Your Day 3 work	Your IOC list and forensic report. If you did not finish, Section 2 gives you everything you need.

You can do this lab even if Day 3 did not go well

Section 2 restates every finding from the Day 3 investigation: the timeline, the indicators, the root cause.

If you finished Day 3, use your own work and treat Section 2 as a check on it.

If you did not, use Section 2 as your starting point. You are not behind, and nothing in Days 4 and 5 is blocked.

1.2 The three websites you will use

Tool	Address	What you use it for
CyberChef	gchq.github.io/CyberChef	Pulling indicators out of large log exports, counting occurrences, and filtering. Published by GCHQ. Runs entirely inside your browser - nothing you type is ever uploaded anywhere.
MITRE ATT&CK	attack.mitre.org	The public catalogue of attacker techniques. You map every step of this intrusion to a technique identifier so other teams can understand it without reading your whole report.
Sigma specification	sigmahq.io	The open, vendor-neutral format for writing detection rules. You write four rules on Day 5. Reference only - you write the rules in a text editor.

If a website will not load

Appendix A.1 prints every ATT&CK technique used in this lab, with its identifier, official name and tactic. Appendix A.2 prints the full Sigma field reference and a complete worked rule.

You can finish this entire lab from the printed appendices. Note in your report which source you used.

You are never blocked. Move on and keep working.

1.3 Scope and ethics

Read this before you open anything

Everything in this lab is evidence that has already been collected for you. You are analysing it, not generating it.

Do not point any scanning tool at any host, website or address anywhere. Not the addresses in this document, not your employer's network, not a site you found online.

The two external addresses in this incident belong to the scenario. Do not visit them, scan them, look them up in a service that will connect to them, or paste them into any site that performs live lookups. Reputation services that actively probe a target are not appropriate here.

The internal addresses are private ranges that do not exist on the public internet.

Analysts get fired and prosecuted for touching things they were not authorised to touch. The habit of checking your authorisation first matters more than any tool in this lab.

2. Where We Left Off

On Day 3 you investigated MTB-WEB01 after the Meridian Trust Bank SOC raised alert SOC-2026-09-02-0416. This is what that investigation established.

2.1 The confirmed intrusion on MTB-WEB01

Time (UTC)	What happened	Technique
02 Sep 02:12:33	Path traversal request returns the contents of /etc/passwd	Exploitation of unpatched Apache 2.4.49
02 Sep 02:15:22	Webshell written to /var/www/html/uploads/style.php	Webshell dropped
02 Sep 02:17:40	Webshell used for the first time	Command execution as www-data
02 Sep 02:21:05	pkexec opens a root session for uid 33 (www-data)	Local privilege escalation
02 Sep 02:23:11	Account svc-backup created with UID 0	Backdoor account
02 Sep 02:24:48	Attacker SSH key added to /root/.ssh/authorized_keys	Persistent remote access
02 Sep 02:25:57	Beacon written to /usr/local/bin/apache-worker	Command and control tooling
02 Sep 02:26:30	Cron entry /etc/cron.d/apache-logrotate runs it every 5 minutes	Persistence
02 Sep 02:34:55	Customer data archived into /tmp/.ICE-unix and later deleted	Collection and staging
02 Sep 02:41:19	Shell history cleared on one account but not the other	Covering tracks

Time (UTC)	What happened	Technique
02 Sep 09:47:12	SOC alerts on the repeated outbound connection	Detection

2.2 The confirmed root cause

Element	Detail	CVE	Day 1-2 finding
Initial access	Apache 2.4.49 path traversal leading to remote code execution	CVE-2021-41773 <i>CVE-2021-42013</i>	MTB-0001 <i>MTB-0002</i>
Enabling misconfiguration	ScriptAlias to /cgi-bin/ combined with Require all granted on the filesystem root	-	Evidence pack
Privilege escalation	polkit pkexec (PwnKit) against policykit-1 0.105-31	CVE-2021-4034	MTB-0003

2.3 The Day 3 indicator list

These are the indicators your forensic report handed to the wider security team. They are your starting point today.

Type	Indicator	Where it was found
Network	185.220.101.47	Beacon destination, exploit source, SSH source
Network	TCP port 8443 outbound	Beacon destination port
File path	/var/www/html/uploads/style.php	Webshell
File path	/usr/local/bin/apache-worker	Beacon
File path	/etc/cron.d/apache-logrotate	Persistence
File path	/tmp/.ICE-unix	Exfiltration staging directory
Account	svc-backup (UID 0)	Backdoor account
SSH key	Key comment attacker@blackfin	/root/.ssh/authorized_keys
User agent	curl/7.81.0	Exploitation requests

The two file hashes recorded at acquisition are reproduced below on their own lines so they can be copied cleanly.

```
/var/www/html/uploads/style.php
2dfc22e2fb93187b7537d9fffec52ecc94fd6416c84b78e68ff7a025e975c20c

/usr/local/bin/apache-worker
1ebcce99569f4337f9ee3746b7b095a9f54abfe3036ef06014b1a00ea4d15d4b
```

What that report did not tell you

A forensic report on one host answers what happened to that host. It does not answer the two questions the bank actually needs answered next.

How long has this been going on? Your timeline started at 02:12 on 2 September because that is when the disk evidence started. That is not the same thing as when the intrusion started.

Where else is the attacker? You examined one server. Meridian Trust runs more than one.

Answering those two questions is Day 4.

3. How to Work Through This Lab

3.1 Time plan

These are guide times, not limits. If a section is taking dramatically longer than the guide, check Appendix A.4 before assuming you are stuck.

Session	Section	What you are doing	Guide time
Day 4	4	Read the hunt brief and the fleet inventory	30 minutes
Day 4	5	Extend the timeline using network telemetry	1 to 2 hours
Day 4	6	Hunt the indicators across the fleet	2 to 3 hours
Day 4	7	Rewrite your indicators using the Pyramid of Pain	1 hour
Day 4	8	Write the incident scoping report	1 to 2 hours
Day 5	9	Map the intrusion to MITRE ATT&CK	1 to 2 hours
Day 5	10	Write four Sigma detection rules	2 to 3 hours
Day 5	11	Containment, eradication and recovery plan	1 to 2 hours
Day 5	12	Post-incident review	1 hour

3.2 Set up your workbook before you start

Where	Name it	What goes in it
Spreadsheet tab 1	Egress Log	The perimeter firewall export from Section 5.1
Spreadsheet tab 2	Fleet Hunt	One row per host, one column per indicator, filled in as you hunt
Spreadsheet tab 3	Extended Timeline	The merged timeline covering the full intrusion, not just 2 September
Spreadsheet tab 4	IOC Tiers	Your rewritten indicator list, sorted by Pyramid of Pain tier
Spreadsheet tab 5	ATT&CK Map	Technique identifier, name, tactic, and your supporting evidence
Text file 1 to 4	mtb_detect_01.yml to mtb_detect_04.yml	Your four Sigma rules, one per file
Document 1	MTB Incident Scoping Report	Your Day 4 deliverable
Document 2	MTB Post-Incident Review - Case MTB-IR-2026-0902	Your Day 5 deliverable

3.3 What a good answer looks like

As on Days 1 to 3, most of the marks are for reasoning. Here is the same hunting conclusion written three ways.

Standard	Example write-up
Not acceptable	MTB-FILE01 is also compromised.
Passing	MTB-FILE01 is also compromised. Its auth log shows an SSH login from MTB-WEB01 at 02:52 on 2 September using the svc-backup account, and it is running a beacon process.
Strong	MTB-FILE01 is compromised with high confidence. Its auth log records an accepted publickey login for svc-backup from 10.50.2.24 (MTB-WEB01) at 02:52:14 on 2 September, twenty-eight minutes after the attacker gained root on MTB-WEB01 and harvested the SSH private key. The process telemetry shows a persistent outbound session to 185.220.102.244:443 attributed to /usr/local/sbin/syslog-forward, held open by a systemd unit rather than a cron entry. Note the tradecraft differs from MTB-WEB01: a different C2 address and a different persistence mechanism. An indicator-only hunt for 185.220.101.47 and /etc/cron.d/apache-logrotate would have missed this host entirely, which is the argument for hunting on behaviour rather than on addresses.

The pattern to copy

State the conclusion and your confidence. Cite the specific log line with its timestamp and source host. Connect it to what you already knew. Note what is different or surprising. Say what that difference means for how the hunt should be run.

Every hunting conclusion in your scoping report should follow that shape.

4. Day 4, Part 1: The Hunt Brief

Your forensic report went to the Head of Security Operations at 08:00 this morning. This came back.

From : Head of Security Operations, Meridian Trust Bank
 To : Incident Response
 Subject : MTB-IR-2026-0902 - scoping required before we brief the CISO

Good report. Two things before this goes any further.

1. I do not accept that this started at 02:12 on 2 September. That is when your disk evidence starts, which is not the same thing. The Apache access log on that image only went back to 1 September because of log rotation. Get me the real start date from the perimeter telemetry, which we retain for 90 days.
2. You examined one server. We run five in that segment. The attacker had root on a host in our DMZ for at least seven hours and there was an SSH private key sitting on it. Assume they used it. Tell me how many hosts are affected, and tell me what you checked on the ones you are calling clean.

I need a scoping report by end of day. Not a full incident report. Scope: how far back, how many hosts, what data.

4.1 What threat hunting actually is

Up to now you have been reactive: an alert fired, and you investigated it. Hunting is the opposite. You start from a hypothesis about what an attacker would do, and you go looking for evidence of it in places nobody has alerted on.

A hunt hypothesis is a specific, testable statement. "Check if we are compromised" is not a hypothesis. "If the attacker harvested the SSH key from MTB-WEB01, other hosts in the server segment will show accepted publickey logins originating from 10.50.2.24 around or after 02:24 on 2 September" is a hypothesis, because it tells you exactly which log, which field and which time window to look at.

Your hypotheses for today	What you will test it against
The intrusion began before 2 September	Perimeter firewall egress log, Section 5.1
The attacker moved from MTB-WEB01 to other hosts using the harvested SSH key	Fleet authentication logs, Section 6.2
Other hosts are running a beacon	Fleet process and network telemetry, Section 6.3
The attacker used the same tooling everywhere	Everything above, and this one is wrong

A hunt that finds nothing is still a result

Two of the five hosts in this environment are genuinely clean. Proving that is as valuable as finding a compromise, and your scoping report must say what you checked on them.

"We found no evidence on MTB-DC01" is worthless on its own. "We reviewed MTB-DC01 authentication logs for the full window, checked for all six file-path indicators and reviewed its egress records, and found no evidence of compromise" is a finding a CISO can act on.

The difference is whether the reader can tell what you would have caught.

4.2 The fleet

These are the five hosts in the server segment, plus the two workstation addresses that appear in the logs. This is your hunting scope.

Host	Address	Role	Status at start of hunt
MTB-WEB01	10.50.2.24	DMZ customer portal	CONFIRMED COMPROMISED (Day 3)
MTB-APP01	10.50.2.25	Application and file transfer	Unknown
MTB-FILE01	10.50.2.26	Internal file server	Unknown
MTB-DC01	10.50.2.10	Windows domain controller	Unknown
MTB-DVWA01	10.50.2.30	Test web application	Unknown
Workstation	10.50.1.31	Analyst workstation, mtbadmin	Seen in Day 3 logs, legitimate
Workstation	10.50.1.44	Analyst workstation	Seen in Day 3 logs, legitimate

5. Day 4, Part 2: Extending the Timeline

Your Day 3 timeline started on 2 September because that is where the disk evidence started. The Apache access log on the image began at 14:02 on 1 September, and the file system timeline could only show you files that still existed when the image was taken.

Neither of those tells you when the attacker first arrived. For that you need telemetry that lives somewhere the attacker could not reach and that is retained longer than the host's own logs.

5.1 The perimeter firewall egress log

Meridian Trust retains 90 days of perimeter connection records. This is the extract for the server segment, filtered to external destinations, covering 26 August to 2 September. Sessions to normal business destinations have been removed.

date	time	src_ip	src_host	dst_ip	dport	proto	action	bytes_out	bytes_in	sessions
2026-08-26	03:41:02	10.50.2.30	MTB-DVWA01	52.168.117.171	443	tcp	allow	1204	880	1
2026-08-28	01:52:17	185.220.101.47	-	10.50.2.24	80	tcp	allow	0	0	1
2026-08-28	01:52:19	185.220.101.47	-	10.50.2.24	80	tcp	allow	0	0	214
2026-08-28	01:58:44	185.220.101.47	-	10.50.2.24	443	tcp	deny	0	0	6
2026-08-28	02:03:11	185.220.101.47	-	10.50.2.24	22	tcp	deny	0	0	3
2026-08-30	02:07:55	185.220.101.47	-	10.50.2.24	80	tcp	allow	0	0	12
2026-08-30	02:09:12	10.50.2.24	MTB-WEB01	185.220.101.47	8443	tcp	allow	842	119	1
2026-08-30	02:14:40	185.220.101.47	-	10.50.2.24	80	tcp	allow	0	0	4
2026-09-01	09:15:03	10.50.2.30	MTB-DVWA01	52.168.117.171	443	tcp	allow	1198	874	1
2026-09-01	09:20:03	10.50.2.30	MTB-DVWA01	52.168.117.171	443	tcp	allow	1201	881	1
2026-09-01	09:25:03	10.50.2.30	MTB-DVWA01	52.168.117.171	443	tcp	allow	1197	879	1
2026-09-02	02:08:14	185.220.101.47	-	10.50.2.24	80	tcp	allow	0	0	9
2026-09-02	02:12:33	185.220.101.47	-	10.50.2.24	80	tcp	allow	0	0	6
2026-09-02	02:24:48	185.220.101.47	-	10.50.2.24	22	tcp	allow	0	0	1
2026-09-02	02:31:12	10.50.2.24	MTB-WEB01	185.220.101.47	8443	tcp	allow	611	88	1
2026-09-02	02:36:12	10.50.2.24	MTB-WEB01	185.220.101.47	8443	tcp	allow	604	88	1
2026-09-02	02:38:02	10.50.2.24	MTB-WEB01	185.220.101.47	8443	tcp	allow	2884416	412	1
2026-09-02	02:41:12	10.50.2.24	MTB-WEB01	185.220.101.47	8443	tcp	allow	598	88	1
2026-09-02	02:57:40	10.50.2.26	MTB-FILE01	185.220.102.244	443	tcp	allow	744	102	1
2026-09-02	03:27:40	10.50.2.26	MTB-FILE01	185.220.102.244	443	tcp	allow	739	102	1
2026-09-02	03:57:40	10.50.2.26	MTB-FILE01	185.220.102.244	443	tcp	allow	741	102	1
2026-09-02	04:27:40	10.50.2.26	MTB-FILE01	185.220.102.244	443	tcp	allow	740	102	1
2026-09-02	05:12:19	10.50.2.26	MTB-FILE01	185.220.102.244	443	tcp	allow	9917722	508	1
2026-09-02	05:27:40	10.50.2.26	MTB-FILE01	185.220.102.244	443	tcp	allow	742	102	1
2026-09-02	09:20:03	10.50.2.30	MTB-DVWA01	52.168.117.171	443	tcp	allow	1203	877	1
2026-09-02	09:25:03	10.50.2.30	MTB-DVWA01	52.168.117.171	443	tcp	allow	1199	883	1

5.2 Reading an egress log

Column	What it tells you
src_ip and src_host	Where the connection came from. A blank src_host means the source is external and not one of ours.
dst_ip and dport	Where it was going. For outbound rows this is the interesting half.
action	Whether the firewall permitted it. A deny row still tells you someone tried.
bytes_out	How much data left the bank. This column is where exfiltration becomes visible.
sessions	How many separate connections this row summarises. A single row with a high session count is a scan, not a conversation.

Three things in this log are worth stopping on

One row has a session count in the hundreds against a single host and port, with zero bytes transferred in either direction. Zero bytes means no conversation actually happened. What activity produces hundreds of connections that carry no data?

Two rows have a bytes_out value thousands of times larger than every other row. Note both, and note which host each one comes from.

One host produces a perfectly regular outbound pattern on a five-minute interval for the entire period, including days before the intrusion. Regularity is a beacon characteristic, so look at it - and then work out why it is not one.

5.3 Step by step

1. Copy the egress log into your **Egress Log** spreadsheet tab. It is space-separated; use your spreadsheet's text-to-columns feature to split it.
2. Sort by date and time. Identify the earliest row involving 185.220.101.47 and record that timestamp. This is your revised start of intrusion.

```
2026-08-28 01:52:17 185.220.101.47 - 10.50.2.24 80 tcp allow 0 0 1
```

3. Compare that date against the start of your Day 3 timeline. Calculate the difference in days. This is how much of the intrusion your disk evidence could not see.

5 day difference

4. Explain in writing why the disk image could not show you this activity. Two reasons apply: one about log rotation, one about where the evidence lives.

Log rotation: host-level logs rotate and age out typically daily or weekly with limited retention. By the time the disk was imaged on Sept 2, any Aug 28 entries had almost certainly already been rotated away and overwritten, so there was nothing left on disk to find even if you'd looked.

Where the evidence lives: this activity never touched the host at all. It's perimeter firewall traffic, recorded and retained on the firewall itself, not on MTB-WEB01's filesystem. A disk image can only ever show you what the host itself wrote to its own disk. Network-level events that the host never logged locally are structurally invisible to disk forensics. You need the network device's own records, which is exactly what this egress log is.

5. Find the row on 30 August where MTB-WEB01 first connects outbound to the attacker. Note what this means: the compromise on 2 September was not the first one.

```
2026-08-30 02:09:12 10.50.2.24 MTB-WEB01 185.220.101.47 8443 tcp allow 842 119 1
```

This is the host itself connecting outbound to the same IP and the same port (8443) that later became the confirmed beacon on Sept 2, three days before anything in the host logs or filesystem timeline begins. This means the September 2nd event is not the initial compromise.

6. Identify every distinct external destination address in the log. There are three. For each, note which internal hosts talked to it.

Destination	Talks to
52.168.117.171	10.50.2.30 (MTB-DVWA01)

185.220.101.47	10.50.2.24 (MTB-WEB01)
185.220.102.244	10.50.2.26 (MTB-FILE01)

7. Sort by bytes_out, largest first. Record the two largest rows with their host, timestamp and volume. Convert the byte counts into megabytes.

2026-09-02 05:12:19	10.50.2.26	MTB-FILE01	185.220.102.244	443	tcp	allow	9917722	508	1
2026-09-02 02:38:02	10.50.2.24	MTB-WEB01	185.220.101.47	8443	tcp	allow	2884416	412	1

9,917,722 bytes = 9.92 MB

2,884,416 bytes = 2.88 MB

8. One of those two large transfers is from a host you have not investigated yet. Note it. You will confirm it in Section 6.

MTB-FILE01. The largest transfer, 9.92 MB to 185.220.102.244, doesn't belong to anything we've examined yet.

5.4 Record: the extended timeline

Date and time (UTC)	Host	Event	Evidence source
2026-08-26 03:41:02	MTB-DVWA01	Baseline outbound connection to 52.168.117.171:443 (1,204 bytes out) - destination recurs throughout the log; not yet attributed to any known-malicious infrastructure	Perimeter egress log (5.1)
2026-08-28 01:52:17	MTB-WEB01	First contact from 185.220.101.47 on port 80, allowed, 214 sessions - five days before any host-level log evidence begins	Perimeter egress log (5.1)
2026-08-28 01:58:44	MTB-WEB01	185.220.101.47 probes port 443 - denied by firewall	Perimeter egress log (5.1)
2026-08-28 02:03:11	MTB-WEB01	185.220.101.47 probes port 22 (SSH) - denied by firewall	Perimeter egress log (5.1)
2026-08-28 02:07:55	MTB-WEB01	185.220.101.47 further port-80 contact, allowed, 12 sessions	Perimeter egress log (5.1)
2026-08-30 02:09:12	MTB-WEB01	Host itself connects OUTBOUND to 185.220.101.47:8443 for the first time (842 bytes out/119 in) - same IP and port later confirmed as C2 destination. Proves 2 Sept was not the first compromise episode; predates all host-log evidence by 3 days	Perimeter egress log (5.1)
2026-08-30 02:14:44	MTB-WEB01	185.220.101.47 contacts port 80 again, allowed, 4 sessions	Perimeter egress log (5.1)
2026-09-01 09:15:23	MTB-DVWA01	Outbound connection to 52.168.117.171:443 (1,198 bytes out) -	Perimeter egress log (5.1)

Date and time (UTC)	Host	Event	Evidence source
		consistent with the recurring baseline pattern	
2026-09-01 09:25:03	MTB-DVWA01	Outbound connection to 52.168.117.171:443 (1,197 bytes out)	Perimeter egress log (5.1)
2026-09-02 02:08:14	MTB-WEB01	Reconnaissance begins: GET /, /server-status, /.git/config, /icons/README from 185.220.101.47 (User-Agent: Nmap Scripting Engine); egress log independently shows a matching allowed port-80 connection, 9 sessions	Apache access log (11.1) + Perimeter egress log (5.1) - matching timestamp
2026-09-02 02:12:33	MTB-WEB01	Path-traversal exploit confirmed: GET /cgi-bin/%2e%2e/%2e%2e/etc/passwd returns 200/622 bytes (curl/7.81.0). Matches CVE-2021-42013 against Apache 2.4.49 (Finding MTB-0001); egress log shows a matching allowed port-80 connection, 6 sessions	Apache access log (11.1) + Perimeter egress log (5.1) - matching timestamp
2026-09-02 02:15:22	MTB-WEB01	Web shell dropped: POST via path-traversal RCE creates /var/www/html/uploads/style.php	Filesystem timeline (11.4) + Apache access log (11.1) - matching timestamp
2026-09-02 02:17:40	MTB-WEB01	Attacker exercises the newly planted web shell: POST /uploads/style.php	Apache access log (11.1)
2026-09-02 02:20:47	MTB-WEB01	Privilege escalation begins: pkexec[1207] opens a root PAM session on behalf of www-data (uid 33) - consistent with CVE-2021-4034 (PwnKit)	Auth log (11.2)
2026-09-02 02:21:05	MTB-WEB01	Root shell executed via pkexec: COMMAND=/bin/sh, CWD=/var/www/html/uploads, matching a web shell POST at the same second	Auth log (11.2) + Apache access log (11.1) - matching timestamp
2026-09-02 02:23:11	MTB-WEB01	Rogue persistence account created: useradd svc-backup with UID=0, GID=0 (root-equivalent); /etc/passwd, /etc/group modified the same second	Auth log (11.2) + Filesystem timeline (11.4) - matching timestamp
2026-09-02 02:24:48	MTB-WEB01	Attacker installs SSH key in /root/.ssh/authorized_keys and logs in directly as root via publickey from 185.220.101.47; egress log independently confirms an allowed port-22 connection from the same IP at the same second	Auth log (11.2) + Filesystem timeline (11.4) + Perimeter egress log (5.1) - matching timestamp
2026-09-02 02:25:57	MTB-WEB01	Payload binary /usr/local/bin/apache-worker written and chmod +x'd from the attacker's root shell	Filesystem timeline (11.4) + root .bash_history (11.4)

Date and time (UTC)	Host	Event	Evidence source
2026-09-02 02:26:30	MTB-WEB01	Cron persistence installed: /etc/cron.d/apache-logrotate written, disguised as routine log rotation; cron RELOAD logged one second later	Filesystem timeline (11.4) + System log (11.3) + root .bash_history (11.4)
2026-09-02 02:31:12	MTB-WEB01	First confirmed C2 beacon connection: outbound to 185.220.101.47:8443 (611 bytes out) - matches apache-worker's first cron firing (PID 1244) at 02:30:01	Perimeter egress log (5.1) + System log (11.3) - matching PID/timing
2026-09-02 02:34:55	MTB-WEB01	Staging directory /tmp/.ICE-unix created; attacker archives /var/www/html/data into /tmp/.ICE-unix/x for exfiltration	Filesystem timeline (11.4) + Apache access log (11.1) + root .bash_history (11.4) - matching timestamp
2026-09-02 02:36:12	MTB-WEB01	Beacon connection continues, 604 bytes out	Perimeter egress log (5.1)
2026-09-02 02:38:02	MTB-WEB01	LARGE OUTBOUND TRANSFER over the beacon connection: 2,884,416 bytes (~2.88 MB) to 185.220.101.47:8443 - consistent with exfiltration of the /tmp/.ICE-unix/x archive staged 3 minutes earlier	Perimeter egress log (5.1)
2026-09-02 02:41:12	MTB-WEB01	Beacon connection continues, 598 bytes out	Perimeter egress log (5.1)
2026-09-02 02:41:19	MTB-WEB01	Anti-forensics: root's .bash_history touched (attempted history -c did not fully clear it - 465 bytes remain); mtbadmin's .bash_history wiped to 0 bytes three seconds later	Filesystem timeline (11.4)
2026-09-02 02:57:40	MTB-FILE01	SECOND HOST COMPROMISED (not yet investigated at host level): first outbound connection to 185.220.102.244:443, same /16 block as the confirmed C2 IP	Perimeter egress log (5.1)
2026-09-02 03:27:40	MTB-FILE01	Recurring beacon-pattern connection to 185.220.102.244:443, ~739 bytes out	Perimeter egress log (5.1)
2026-09-02 03:57:40	MTB-FILE01	Recurring beacon-pattern connection, ~741 bytes out	Perimeter egress log (5.1)
2026-09-02 04:27:40	MTB-FILE01	Recurring beacon-pattern connection, ~740 bytes out	Perimeter egress log (5.1)
2026-09-02 05:12:19	MTB-FILE01	LARGEST TRANSFER IN THE ENTIRE LOG: 9,917,722 bytes (~9.92 MB) to 185.220.102.244:443 - likely exfiltration event	Perimeter egress log (5.1)

Date and time (UTC)	Host	Event	Evidence source
2026-09-02 05:27:40	MTB-FILE01	Beacon-pattern connection continues post-transfer, ~742 bytes out	Perimeter egress log (5.1)
2026-09-02 09:20:01	MTB-DVWA01	Outbound connection to 52.168.117.171:443 (1,203 bytes out)	Perimeter egress log (5.1)
2026-09-02 09:25:03	MTB-DVWA01	Outbound connection to 52.168.117.171:443 (1,199 bytes out)	Perimeter egress log (5.1)
2026-09-02 09:47:12	MTB-WEB01	SOC alert SOC-2026-09-02-0416 raised: sustained 5-minute-interval outbound beaconing from MTB-WEB01 to 185.220.101.47:8443, non-allowlisted destination; rogue apache-worker process identified	SOC alert (8.1)

Merge this with your Day 3 timeline rather than keeping them separate. The report needs one timeline covering the whole intrusion.

6. Day 4, Part 3: Hunting the Fleet

You now know the intrusion is older than you thought and that at least one other host has been talking to attacker infrastructure. This section is where you confirm which hosts are affected and, just as importantly, which are not.

6.1 How to run a hunt across many hosts

Build a grid. One row per host, one column per indicator. Fill in every cell, including the ones where the answer is no. The empty cells at the end are what tells you where you still have no visibility.

Host	185.220.101.47	185.220.102.244	apache-worker	cron entry	svc-backup	syslog-forward	systemd persistence	Lateral SSH from WEB01/FILE01	Verdict
MTB-WEB01	YES	NO	YES	YES	YES	NO	NO	N/A (origin host)	COMPROMISED
MTB-APP01	NO	NO	NO		NO	NO		NO	CLEAN
MTB-FILE01	NO	YES	NO		YES	YES	YES	YES	COMPROMISED
MTB-DC01	NO	NO	N/A (Windows)	N/A (Windows)		N/A (Windows)		NO	CLEAN
MTB-DVWA01	NO	NO	NO	NO	NO	NO	NO	NO	CLEAN

The trap in this grid

If you fill in only these columns you will reach the wrong answer on one host, because the attacker did not use the same tooling everywhere.

When you finish the grid, add columns for whatever new indicators you discover along the way. A hunt grid that never grows is a hunt that only found what it already knew about.

6.2 Fleet authentication logs

Authentication records for the four uninvestigated hosts, covering 2 September. Routine internal activity has been left in so you can see what normal looks like.

MTB-APP01 (10.50.2.25)

```
Sep  2 07:02:11 mtb-app01 sshd[2201]: Accepted publickey for mtbadmin from 10.50.1.31 port 51002
Sep  2 07:44:56 mtb-app01 sshd[2240]: Accepted publickey for mtbdeploy from 10.50.1.44 port 39114
Sep  2 11:20:03 mtb-app01 sshd[2388]: Failed password for invalid user oracle from 10.50.1.99 port 44120
Sep  2 11:20:07 mtb-app01 sshd[2388]: Failed password for invalid user oracle from 10.50.1.99 port 44120
Sep  2 11:20:12 mtb-app01 sshd[2388]: Connection closed by invalid user oracle 10.50.1.99
```

MTB-FILE01 (10.50.2.26)

```
Sep  2 06:25:01 mtb-file01 CRON[1801]: pam_unix(cron:session): session opened for user root(uid=0)
Sep  2 02:52:14 mtb-file01 sshd[1655]: Accepted publickey for svc-backup from 10.50.2.24 port 52118
Sep  2 02:52:14 mtb-file01 sshd[1655]: pam_unix(sshd:session): session opened for user svc-backup(uid=0)
Sep  2 02:54:39 mtb-file01 useradd[1668]: new user: name=svc-backup, UID=0, GID=0, home=/home/svc-backup
Sep  2 02:54:39 mtb-file01 useradd[1668]: WARNING: UID 0 is already in use by user root
Sep  2 02:56:02 mtb-file01 systemd[1]: Reloading.
Sep  2 02:56:03 mtb-file01 systemd[1]: Started System Log Forwarder.
Sep  2 08:11:47 mtb-file01 sshd[1902]: Accepted publickey for mtbadmin from 10.50.1.31 port 49882
```

MTB-DC01 (10.50.2.10)

Timestamp	Event ID	Logon Type	Account	Source
09/02/2026 07:14:22	4624	Logon Type 3	Account: MTB\\mtbadmin	Source: 10.50.1.31
09/02/2026 07:51:09	4624	Logon Type 2	Account: MTB\\Administrator	Source: 10.50.2.10
09/02/2026 09:02:44	4624	Logon Type 3	Account: MTB\\svc-sql	Source: 10.50.2.25
09/02/2026 13:30:17	4625	Logon Type 3	Account: MTB\\jsmith	Source: 10.50.1.44
09/02/2026 13:30:41	4624	Logon Type 3	Account: MTB\\jsmith	Source: 10.50.1.44

MTB-DVWA01 (10.50.2.30)

```
Sep  2 08:40:15 mtb-dvwa01 sshd[3301]: Accepted publickey for mtbdeploy from 10.50.1.44 port 40221
Sep  2 15:02:38 mtb-dvwa01 sshd[3455]: Accepted publickey for mtbdeploy from 10.50.1.44 port 40990
```

6.3 Fleet process and network telemetry

Endpoint telemetry showing long-running processes holding external network sessions, captured at 10:20 on 2 September across all five hosts.

host	pid	user	process	parent	remote_endpoint	state	uptime
MTB-WEB01	1244	root	/bin/bash /usr/local/bin/apache-worker	cron	185.220.101.47:8443	ESTAB	07:49

MTB-WEB01	901	root	/usr/sbin/apache2 -k start	systemd	-	-	8d
MTB-APP01	2105	root	/usr/sbin/vsftpd	systemd	-	-	8d
MTB-APP01	2110	root	/usr/sbin/sshd -D	systemd	-	-	8d
MTB-FILE01	1671	root	/usr/local/sbin/syslog-forward	systemd	185.220.102.244:443	ESTAB	07:24
MTB-FILE01	1520	root	/usr/sbin/smbd -D	systemd	-	-	8d
MTB-DC01	4188	SYSTEM	C:\Windows\System32\svchost.exe	services	-	-	8d
MTB-DC01	4402	SYSTEM	C:\Windows\System32\lsass.exe	wininit	-	-	8d
MTB-DVWA01	3102	monagent	/opt/monagent/bin/monagent --daemon	systemd	52.168.117.171:443	ESTAB	31d
MTB-DVWA01	3010	root	/usr/sbin/apache2 -k start	systemd	-	-	8d

6.4 The asset register entry for MTB-DVWA01

Before you make a judgement on the host with the regular five-minute outbound pattern, read its asset record.

```
Asset:          MTB-DVWA01
Owner:          Application Security - test environment
Environment:    Non-production, isolated test
Installed agents: monagent 4.2.1 (Meridian Trust estate monitoring)
                  Check-in interval: 300 seconds
                  Check-in endpoint: 52.168.117.171:443 (approved egress allowlist)
Deployed:       2026-08-02
```

6.5 Step by step

1. Fill in the grid in Section 6.1 for all five hosts, using the logs and telemetry above. Every cell gets an answer.
2. Identify the host that shows an SSH login originating from MTB-WEB01. Record the exact timestamp, the account used, and how long after the MTB-WEB01 root compromise it occurred.

```
Sep  2 02:52:14 mtb-file01 sshd[1655]: Accepted publickey for svc-backup from 10.50.2.24 port 52118
```

Timestamp: 02:52:14 UTC

Account used: svc-backup

Time after root compromise on WEB01: root compromise on WEB01 occurred at 02:21:05 (the pkexec root shell).
02:52:14 – 02:21:05 = 31 minutes, 9 seconds later.

3. Explain how the attacker was able to authenticate to that second host. Your Day 3 evidence tells you what was sitting on MTB-WEB01 that made this possible.

At 02:24:50, the filesystem timeline shows /home/svc-backup/.ssh/authorized_keys written on WEB01, the attacker's backup SSH key. FILE01's login is publickey auth for the same account name (svc-backup), succeeding with no password prompt.

4. On that second host, identify the beacon. Note three ways it differs from the one on MTB-WEB01: the destination address, the file path, and the persistence mechanism.

Destination:

MTB-WEB01: 185.220.101.47:8443

MTB-FILE01: 185.220.102.244:443

Difference: Different IP and port

File Path:

MTB-WEB01: /usr/local/bin/apache-worker

MTB-FILE01: /usr/local/sbin/syslog-forward

Difference: Different directory and different disguise name

Persistence Mechanism:

MTB-WEB01: Cron (/etc/cron.d/apache-logrotate)

MTB-FILE01: systemd unit

Difference: Cron job vs. a persistent systemd service

5. Add columns to your grid for the new indicators you just found, and re-check every host against them.
6. Look at MTB-DVWA01. It has a persistent external session on a regular five-minute interval, which is exactly the pattern that got MTB-WEB01 flagged. Decide whether it is compromised, and write down the specific evidence that settles it.

Asset register shows this the 5-minute interval is normal:

- monagent 4.2.1, Meridian Trust's own estate-monitoring agent, deployed 2026-08-02
 - Check-in interval: 300 seconds to 52.168.117.171:443, explicitly listed as the approved egress allowlist destination for this agent
 - Auth log shows only mtbdeploy logins from 10.50.1.44, a known deployment host
7. For the two hosts you conclude are clean, write down exactly what you checked. Not "nothing found" - list the indicators and the log sources.

MTB-APP01

- Network: no connections to 185.220.101.47 or 185.220.102.244 in the egress log (5.1)
- Process telemetry (6.3): only /usr/sbin/sshd -D and /usr/sbin/vsftpd -D. No rogue process, no established external session
- Auth log (6.2): logins only from known hosts (mtbadmin from 10.50.1.31, mtbdeploy from 10.50.1.44); no svc-backup account creation; no login sourced from 10.50.2.24 (WEB01) or 10.50.2.26 (FILE01)
- The one anomaly present (failed oracle logins) is addressed separately in Q8

MTB-DC01

- Network: no entries at all as a source in the egress log (5.1)
 - Process telemetry (6.3): only svchost.exe and lsass.exe, both standard, no remote_endpoint/ESTAB state on either
 - Windows Security log (6.2): all four logon events sourced from known internal IPs (10.50.1.31, 10.50.2.10 itself, 10.50.2.25, 10.50.1.44), none from 10.50.2.24 or 10.50.2.26
8. Look at the failed logins on MTB-APP01 at 11:20. Decide whether they are related to this incident and justify your answer using the source address and the timing.

Unrelated to this incident:

- Source address: 10.50.1.99 doesn't appear anywhere else in this entire investigation. Not 185.220.101.47, not 185.220.102.244, not either compromised host (10.50.2.24, 10.50.2.26). It sits on the 10.50.1.x segment, the same range as the legitimate admin/deploy hosts, not the compromised 10.50.2.x segment
- Timing: 11:20 is roughly 8.5 hours after the intrusion's active window (02:08–02:41) and about 1.5 hours after the SOC alert (09:47), well outside the attack's established timeline, with no other activity bridging the two.

- Technique mismatch: this incident's actual TTPs used planted SSH keys and account creation for lateral movement, never password-guessing. Three rapid password attempts against a nonexistent oracle account reads as a generic/automated scan hitting a stale legacy account name, not a targeted continuation of this attack chain.

The point of the MTB-DVWA01 question

Periodicity is a beacon characteristic. It is not proof of one. Plenty of legitimate software checks in on a fixed schedule, and every monitoring agent in your estate looks like a beacon if you squint.

What separates them is not the pattern. It is whether the process, the destination and the schedule are all accounted for by something you approved.

An analyst who cannot tell the difference generates false alarms until people stop reading their alerts. That is a worse outcome than missing one beacon.

7. Day 4, Part 4: Rewriting Your Indicators

Your hunt just demonstrated something uncomfortable. The indicator list you produced on Day 3 would have missed MTB-FILE01 completely, because the attacker used a different address, a different filename and a different persistence mechanism on that host.

That is not a failure of your investigation. It is a property of indicators, and there is a well-known model for thinking about it.

7.1 The Pyramid of Pain

Introduced by David Bianco in 2013, the Pyramid of Pain ranks indicator types by how much difficulty you cause an attacker when you detect and block them. The higher up you can detect, the harder it is for the attacker to carry on.

Tier	Indicator type	Pain caused	Example from this incident
6 (top)	TTPs	Tough	A beacon process launched by a persistence mechanism, holding a long-lived outbound session on a fixed interval
5	Tools	Challenging	The specific beacon implementation, whichever filename it is given
4	Network or host artifacts	Annoying	A root-owned process in /usr/local/ with a persistence entry created outside a change window
3	Domain names	Simple	(none observed in this incident)
2	IP addresses	Easy	185.220.101.47 and 185.220.102.244
1 (bottom)	Hash values	Trivial	The SHA-256 of style.php and apache-worker

Why this incident proves the model

You blocked 185.220.101.47 after Day 3. The attacker was already using 185.220.102.244 on the second host, at no cost to them - changing an address takes seconds.

You hashed apache-worker. On MTB-FILE01 the same capability is called syslog-forward, which produces a different hash.

What did not change between the two hosts: a root-owned binary outside the package manager's control, started by a persistence mechanism, holding a long-lived outbound session at a fixed interval. Detect that and the attacker has to redesign their tooling, not edit a config file.

This is why Day 5 asks you to write detections for behaviour rather than for addresses.

7.2 Step by step

1. List every indicator from Day 3 and every new one from today. There should be at least sixteen.
2. Assign each one a Pyramid tier from the table in Section 7.1.
3. For each indicator at tier 1 or 2, write down how long it would take the attacker to change it and carry on. Be honest - the answer is usually seconds or minutes.
4. Write three tier 4 to 6 descriptions of this intrusion that do not name a single address, filename or hash. These become the basis of your Day 5 detection rules, so make them specific enough to test.
5. Explain in two or three sentences why an indicator list is still worth producing even though most of it sits at the bottom two tiers.

An indicator list is still worth producing because tier 1 and 2 indicators, even though they're trivial for the attacker to change, are cheap and fast for a defender to check right now. They let another team search their own environment today and get an immediate yes/no on whether they've been hit by this specific actor's current infrastructure. The list is also the evidentiary record: it ties the tier 4–6 findings back to concrete, timestamped, sourced facts, rather than leaving the report as an unsubstantiated claim about "an attacker" with no way to verify it.

7.3 Record: tiered indicator list

Tier	Indicator or behaviour description	Source	Attacker cost to change
1	SHA-256 of style.php (web shell)	Recovered file (11.4) + Artefact hashes	Seconds
1	SHA-256 of apache-worker (WEB01 payload)	Artefact hashes	Seconds
1	SHA-256 of /etc/cron.d/apache-logrotate	Artefact hashes	Seconds
2	IP 185.220.101.47	Access log, auth log, egress log	Minutes
2	IP 185.220.102.244	Egress log, FILE01 auth log	Minutes
4	File path /var/www/html/uploads/style.php	Filesystem timeline, access log	
4	File path /usr/local/bin/apache-worker	Filesystem timeline, .bash_history, process telemetry	
4	File path /usr/local/sbin/syslog-forward	Process telemetry (6.3)	

Tier	Indicator or behaviour description	Source	Attacker cost to change
4	File /etc/cron.d/apache-logrotate (cron persistence)	Filesystem timeline, system log	
4	Attacker key in /root/.ssh/authorized_keys	Filesystem timeline, recovered file	
4	Attacker key in /home/svc-backup/.ssh/authorized_keys	Filesystem timeline	
4	svc-backup account, UID=0/GID=0	Auth log (WEB01 + FILE01)	
4	Staging directory /tmp/.ICE-unix	Filesystem timeline, .bash_history	
4	Systemd persistence unit "Log Forwarder" (FILE01)	Auth log (6.2), reload events	
4	Outbound port 8443 (WEB01 beacon)	Volatile capture, egress log	
4	Outbound port 443 (FILE01 beacon)	Process telemetry (6.3), egress log	
5	The beacon tool itself, same underlying implementation reused under two different filenames (apache-worker, syslog-forward)	Process telemetry, filesystem timeline	
6	TTP: unauthenticated path-traversal → RCE → local pkexec/PwnKit privilege escalation to root	Full chain, 11.1–11.2	
6	TTP: lateral movement by reusing a planted SSH key across a shared authorized_keys trust path, disguised under a plausible service-account name	WEB01 + FILE01 auth logs	
6	TTP: disguising persistence as routine sysadmin artifacts (cron "log rotation," systemd "log forwarder") to blend into normal operations	Cron/systemd entries, both hosts	

8. Day 4, Part 5: The Incident Scoping Report

The Head of Security Operations asked three questions: how far back, how many hosts, what data. Answer those three and nothing else. This is not a second forensic report.

8.1 What goes in it

1. State the revised start of intrusion, with the evidence, and state how many days earlier it is than your Day 3 timeline suggested. Explain in one sentence why the disk evidence could not show this.
2. State the number of compromised hosts and name them, each with the evidence that confirms it and your confidence level.

3. State which hosts you assess as clean, and for each one list what you checked. A reader must be able to tell what your check would and would not have caught.
4. State the data at risk. Two large outbound transfers appear in the egress log. Give the host, timestamp and volume of each, and say what you believe left the bank based on what each host stores.
5. State the lateral movement path, from initial access through to the second host, naming the mechanism at each step.
6. State what you still do not know, and what evidence you would need to close each gap. A scoping report that claims certainty it does not have is worse than one that names its blind spots.
7. Write a short executive summary, three to five sentences, that answers the three questions in plain language.

Dwell time is the number that will be quoted

Dwell time is the gap between the attacker's first activity and the moment you detected them. It is the single figure that gets repeated to the board, the regulator and the auditor, because it measures the security programme rather than the attacker.

Calculate it from your revised start date, not from the disk timeline. Show your working in the report, because the first thing anyone senior will ask is how you arrived at that date.

8.2 Report structure to follow

Section	Contents
Executive summary	Three to five sentences answering how far back, how many hosts, what data.
Revised timeline	Start of intrusion with evidence, dwell time, and why it differs from the Day 3 report.
Confirmed compromised hosts	Each host, the evidence, your confidence, and the tradecraft observed on it.
Hosts assessed as clean	Each host, and exactly what was checked against what indicators.
Lateral movement	The path from initial access to the second host, mechanism by mechanism.
Data at risk	The two large transfers, with volumes, and what each host holds.
Intelligence gaps	What you do not know, and what would close it.

Executive Summary

On September 2, 2026, an external attacker exploited a known, unpatched vulnerability in Meridian Trust's DMZ web server to gain an initial foothold, then escalated to full administrator control and used that access to move laterally into a second host on the internal network. First contact with the attacker's infrastructure actually dates back to August 28, five days before any host-level evidence begins, putting real dwell time well beyond what the original Day 3 investigation captured. Two confirmed hosts (the web server and the fleet file server) show direct evidence of compromise, including persistent backdoors, a disguised root-equivalent account, and outbound data transfers totalling roughly 12.8 MB to attacker-controlled infrastructure. The compromised web server has been isolated from the network; the file server has not yet been contained and remains an active risk. Two other hosts show no evidence of compromise against every indicator checked, and one host, the domain controller, still lacks the evidence needed to close it out.

Revised Timeline

(Refer to 5.4 “Record: the extended timeline”)

Confirmed Compromised Hosts

MTB-WEB01 (10.50.2.24)

Evidence: Web shell (style.php) planted via path-traversal RCE; pkexec privilege escalation from www-data to root; rogue svc-backup account created with UID=0; attacker SSH keys installed on both root and svc-backup; persistence via /etc/cron.d/apache-logrotate; beacon process apache-worker holding a live connection to 185.220.101.47:8443; anti-forensic history-clearing (partial on root, complete on mtbadmin).

Confidence: High. Every stage independently corroborated across at least two evidence sources (filesystem timeline, access log, auth log, system log, or egress log) with matching timestamps.

Tradecraft observed: Exploits a public CVE for initial access, chains an immediate local privilege-escalation exploit rather than waiting, disguises all persistence as mundane sysadmin artifacts (log rotation, a plausibly-named service binary), and makes a partial, imperfect attempt to cover its tracks.

MTB-FILE01 (10.50.2.26)

Evidence: Lateral SSH login as svc-backup originating from MTB-WEB01 (10.50.2.24) at 02:52:14; a second UID=0 svc-backup account created locally 2 minutes 25 seconds later, same warning signature as WEB01; persistence via a systemd unit ("Log Forwarder") rather than cron; beacon process syslog-forward holding a live connection to 185.220.102.244:443; a 9.92 MB outbound transfer at 05:12:19, the single largest transfer in the entire egress log.

Confidence: High on the compromise itself; Medium on full scope, since no filesystem timeline or recovered-file evidence exists for this host the way it does for WEB01. Everything currently rests on auth log, process telemetry, and egress log.

Tradecraft observed: Reuses the exact account-creation pattern and account name from WEB01, but deliberately varies the mechanism (systemd instead of cron) and the binary name/path (syslog-forward in /usr/local/sbin instead of apache-worker in /usr/local/bin). Consistent with an attacker who understands that identical IOCs across hosts are what gets caught, and is varying them on purpose.

Hosts Assessed as Clean

MTB-APP01 (10.50.2.25) checked against:

Egress log (5.1): no connections to 185.220.101.47 or 185.220.102.244

Process telemetry (6.3): only sshd and vsftpd, no rogue process, no established external session

Auth log (6.2): logins only from known internal hosts (mtbadmin from 10.50.1.31, mtbdeploy from 10.50.1.44); no svc-backup; no login sourced from either compromised host

One unrelated anomaly (failed oracle logins from 10.50.1.99 at 11:20) assessed separately and ruled unconnected to this incident on source, timing, and technique grounds

MTB-DVWA01 (10.50.2.30) checked against:

Egress log (5.1): recurring connections to 52.168.117.171:443 initially flagged, since this matched the same "consistent interval" pattern that got WEB01 caught

Asset register (6.4): confirmed as monagent v4.2.1's approved, allowlisted 300-second check-in destination, deployed 2026-08-02, a full month before the incident

Process telemetry (6.3): /opt/monagent/bin/monagent --daemon, uptime 31 days, launched normally via systemd

Auth log: only routine mtbdeploy logins

No overlap with any confirmed indicator from this incident

Lateral Movement

Path: MTB-WEB01 → MTB-FILE01

Mechanism, step by step:

1. 02:24:50: Attacker, already root on WEB01, writes their SSH key into /home/svc-backup/.ssh/authorized_keys on WEB01.
2. ~02:52:14: That same key authenticates svc-backup via SSH from 10.50.2.24 (WEB01) to MTB-FILE01. This only works because FILE01 already trusts that key, most likely because /home is shared/mounted between the two hosts, letting a credential planted on one host work on the other without any additional exploit. This is inferred from the evidence rather than directly confirmed; the actual mount configuration hasn't been checked.
3. 02:54:39: Attacker replicates the WEB01 playbook on FILE01: creates a local svc-backup account with UID=0/GID=0 (same useradd warning about UID 0 already being in use).
4. ~02:56:02–03: Persistence installed via a systemd unit ("Log Forwarder"), reloaded and started. The mechanism differs from WEB01's cron-based approach.
5. Beacon (syslog-forward) established outbound to 185.220.102.244:443, live at time of capture.

Data at Risk

MTB-WEB01 - 2.88MB exfiltrated to 185.220.101.47:8443

This host holds web application root and site data

MTB-FILE01 - 9.92MB exfiltrated to 185.220.102.244:443

This host holds file-server contents

Intelligence Gaps

Gap	What would close it
What was actually inside the FILE01 exfil transfer?	A disk image of MTB-FILE01, matched against a filesystem timeline the same way WEB01's /tmp/.ICE-unix/x was matched.
Whether MTB-DC01 is genuinely clean	A full local-account audit and scheduled-task/service enumeration on the domain controller.
Whether /home is actually shared between WEB01 and FILE01	Direct confirmation via /etc/fstab, NFS exports, or Samba config on both hosts.
Full extent of the August 28–September 2 window	The perimeter egress log only shows connection metadata for this period; no host-level logs survive from before Sept 2 due to log rotation, so anything the attacker did on WEB01 in that five-day window before the confirmed timeline starts is currently invisible.
Attacker identity/infrastructure beyond the two observed IPs	Threat-intel enrichment on 185.220.101.47 and 185.220.102.244 (hosting provider, historical activity,

	any public attribution), and investigation of the attacker@blackfin SSH key comment as a possible handle.
Whether other hosts trust the same shared SSH key path	Fleet-wide check of authorized_keys files and shared-mount configuration beyond just WEB01 and FILE01.

9. Day 5, Part 1: Mapping the Intrusion to MITRE ATT&CK

You have described this intrusion twice now: once as a forensic timeline and once as a scoping report. Both are written in Meridian Trust's own words. That is a problem the moment you need to talk to anyone outside your team.

MITRE ATT&CK is the shared vocabulary that solves it. It is a free, public catalogue of the things attackers actually do, each with a stable identifier. When you say T1505.003, every analyst, every vendor and every threat intelligence report means the same thing by it.

9.1 How ATT&CK is organised

Level	What it is	Example
Tactic	The attacker's goal at that moment. Why they are doing it. Fifteen of them, numbered TA####.	Persistence (TA0003)
Technique	How they achieve that goal. Numbered T####.	Server Software Component (T1505)
Sub-technique	A specific variant of a technique. Numbered T####.###.	Web Shell (T1505.003)

One thing that will confuse you if nobody warns you

ATT&CK renamed a tactic. What older training material, older tooling and most blog posts still call Defense Evasion is now called Stealth (TA0005).

There is also a newer tactic called Defense Impairment (TA0112) that did not previously exist.

Use the names as they appear on attack.mitre.org today, and if you are reading an older report that says Defense Evasion, understand it means the same thing. Appendix A.1 lists the current tactic names.

9.2 Looking up a technique

1. Go to attack.mitre.org. The front page shows the Enterprise matrix: tactics across the top, techniques listed underneath each one.
2. To go straight to a technique, use the address attack.mitre.org/techniques/T1190/ and swap in the identifier you want.
3. For a sub-technique, the identifier splits across the path. T1505.003 is at attack.mitre.org/techniques/T1505/003/. This catches everyone once.
4. On the technique page, read the description and check it genuinely matches what you observed. Record the identifier, the official name and the tactic.

- Read the **Detection** section near the bottom of the page. It describes what data sources reveal this technique. You will use this in Section 10 when you write your rules.

9.3 Build the mapping

Below is the mapping, partly filled in. Complete the empty rows from your Day 3 and Day 4 evidence. Every row needs a piece of evidence you can point at.

Tactic	Technique ID	Technique name	Your evidence
Reconnaissance	T1595.002	Vulnerability Scanning	Egress log, 28 Aug 01:52, 214 sessions to port 80 carrying zero bytes
Initial Access	T1190	Exploit Public-Facing Application	Access log, 02:12:33, GET /cgi-bin/%2e%2e/%2e%2e/etc/passwd → 200/622 bytes (curl/7.81.0). Matches CVE-2021-42013 against Apache 2.4.49-1ubuntu4 (Finding MTB-0001)
Persistence	T1505.003	Web Shell	Filesystem timeline, /var/www/html/uploads/style.php created 02:15:22 via traversal RCE POST; recovered file content confirms PHP web shell disguised as a stylesheet
Privilege Escalation	T1068	Exploitation for Privilege Escalation	Auth log, 02:20:47–02:21:05, pkexec[1207] opens a root session on behalf of www-data (uid 33), executes /bin/sh as root from CWD=/var/www/html/uploads. Consistent with CVE-2021-4034 (PwnKit)
Persistence	T1136.001	Local Account	Auth log, 02:23:11 WEB01: useradd svc-backup, UID=0, GID=0; repeated on FILE01, 02:54:39, identical UID=0 warning signature
Persistence	T1098.004	SSH Authorized Keys	Filesystem timeline, /root/.ssh/authorized_keys written 02:24:48; /home/svc-backup/.ssh/authorized_keys written 02:24:50; recovered content shows attacker key commented attacker@blackfin
Persistence	T1053.003	Cron	Filesystem timeline + system log, /etc/cron.d/apache-logrotate written 02:26:30; cron[812]: RELOAD logged 02:26:31
Persistence	T1543.002	Systemd Service	FILE01 auth log, systemd[1]: Reloading / Started Log Forwarder at 02:56:02–03; process telemetry shows /usr/local/sbin/syslog-forward launched by systemd, parent - (daemon)

Tactic	Technique ID	Technique name	Your evidence
Credential Access	T1552.004	Private Keys	No direct evidence found
Lateral Movement	T1021.004	SSH	FILE01 auth log, 02:52:14, Accepted publickey for svc-backup from 10.50.2.24 port 52118. WEB01's own IP, using the key planted at 02:24:50
Collection	T1560.001	Archive via Utility	Root .bash_history: tar czf /tmp/.ICE-unix/x /var/www/html/data; filesystem timeline confirms /tmp/.ICE-unix created 02:34:55
Command and Control	T1071.001	Web Protocols	Volatile capture + process telemetry: apache-worker (WEB01) ESTABLISHED to 185.220.101.47:8443; syslog-forward (FILE01) ESTABLISHED to 185.220.102.244:443
Exfiltration	T1041	Exfiltration Over C2 Channel	Egress log: WEB01, 02:38:02, 2,884,416 bytes to 185.220.101.47:8443 (same port/destination as the beacon); FILE01, 05:12:19, 9,917,722 bytes to 185.220.102.244:443 (same port/destination as its beacon)
Stealth	T1070.003	Clear Command History	Filesystem timeline, root .bash_history touched 02:41:19 (history -c recovered in content, only partially effective; 465 bytes remained); mtbadmin/.bash_history wiped to 0 bytes at 02:41:22
Discovery	T1082	System Information Discovery	Root .bash_history (recovered file, 11.4): id, uname -a, cat /etc/os-release, which curl wget python3. The attacker's first actions after gaining the root shell, checking the OS and available tooling before building persistence
Persistence / Lateral Movement	T1078.003	Valid Accounts: Local Accounts	WEB01 auth log, 02:24:48, root logs in via SSH using the planted key; no exploit required, just a credential; FILE01 auth log, 02:52:14, svc-backup authenticates via SSH the same way. Both are the attacker using an account as a legitimate credential rather than exploiting anything

9.4 Step by step

1. Complete the evidence column for all fifteen rows. Each entry needs a host, a timestamp and a source.
2. Two techniques in the table appear on both compromised hosts but with different implementations. Identify them and note both implementations.

3. One technique in the table is evidenced by something you decoded rather than something you observed running. Identify it.
4. Add at least two techniques that are not in the table but that you have evidence for. Appendix A.1 lists the full set used in this incident if you want to check your work afterwards - try it yourself first.
5. For each of the fifteen ATT&CK tactics, note whether this attacker used it. The ones they did not use are as informative as the ones they did.

The test of a good mapping

Hand your completed table to someone who has not read your reports. They should be able to describe the intrusion from it alone, in order, without asking you a question.

If they cannot, you have mapped labels rather than an attack.

10. Day 5, Part 2: Writing Detections

Your Day 3 report recommended better monitoring. Your Day 4 hunt showed exactly why the obvious detections would not have worked: block one address and the attacker uses another, hash one file and the next one has a different name.

This section turns the recommendation into something a SOC can actually deploy.

10.1 What Sigma is

Sigma is an open, vendor-neutral format for writing detection rules. A rule is a small YAML file describing what to look for. Converters then translate that one rule into a Splunk search, a Sentinel KQL query, an Elastic query or whatever the organisation runs.

This matters because detection logic written directly in a vendor's query language is locked to that vendor. Meridian Trust has changed SIEM twice in six years. The Sigma rules survived both migrations; the hand-written searches did not.

10.2 The parts of a rule

Field	Required?	What it holds
title	Mandatory	A short description of what the rule detects. Keep it under about 50 characters.
logsource	Mandatory	Which logs to run against. Built from category, product and service.
detection	Mandatory	What to match. Contains one or more named search identifiers plus a condition.
condition	Mandatory	Inside detection. How the named searches combine, for example 'selection and not filter'.
id	Optional	A UUID so the rule can be referenced. Generate one or leave it out for this lab.
status	Optional	One of stable, test, experimental, deprecated, unsupported.
description	Optional	Fuller explanation, including why the rule exists.
references	Optional	Links justifying the rule, for example the ATT&CK technique page.
author	Optional	Who wrote it.

Field	Required?	What it holds
date	Optional	ISO 8601 format, YYYY-MM-DD. Not any other format.
tags	Optional	Namespaced and lower case, for example attack.t1505.003.
falsepositives	Optional	What legitimate activity could also match. Never leave this empty in practice.
level	Optional	One of informational, low, medium, high, critical.

Two rules about YAML that will cost you time if you ignore them

Indentation is meaningful and must be spaces, never tabs. Four spaces per level is the Sigma convention. If your editor inserts tabs, turn that off before you start.

A colon followed by a space separates a key from its value. A colon inside a value, such as in a file path or a URL, must be inside quotes or YAML will try to read it as another key.

10.3 A complete worked rule

This is a full, valid Sigma rule detecting the backdoor account creation you found on both compromised hosts. Read every line before you write your own.

```

title: Linux Local Account Created With UID 0
id: 8f2c1a44-3b7e-4d19-9c02-5ae6f1d0b731
status: experimental
description: |
    Detects creation of a Linux local account assigned UID 0. Such an account
    holds full root privilege while appearing in /etc/passwd as an ordinary
    service account, and useradd logs a warning when it happens.
    Observed on MTB-WEB01 and MTB-FILE01 during case MTB-IR-2026-0902.
references:
  - https://attack.mitre.org/techniques/T1136/001/
author: Meridian Trust Bank SOC
date: 2026-09-04
tags:
  - attack.t1136.001
  - attack.t1078.003
logsource:
  product: linux
  service: auth
detection:
  keywords:
    - 'UID 0 is already in use'
  condition: keywords
falsepositives:
  - Deliberate creation of a second root-equivalent account under a documented
    break-glass procedure. Should be rare and covered by a change record.
level: critical

```

10.4 Why that rule is a good one

- **It sits high on the Pyramid.** It does not name an account, an address or a file. Any attacker creating a UID 0 account on any host trips it. To evade it they must stop using that technique.
- **It fires on the attacker's own noise.** The operating system generates that warning by itself. The attacker cannot suppress it without also failing to create the account.

- **Its false positive case is named and realistic.** A break-glass account is a real thing that would match. Saying so tells the SOC how to triage a hit rather than leaving them guessing.
- **It would have fired twice in this incident,** at 02:23:11 on MTB-WEB01 and again at 02:54:39 on MTB-FILE01, the second one 53 minutes before any alert actually reached a human.

10.5 Write four rules

Write one rule per numbered requirement, saved as a separate .yaml file. Use the worked rule as your template.

Rule	What it must detect	Evidence it must catch
mtb_detect_01	A persistence entry, cron or systemd, that executes a binary from /usr/local/	/etc/cron.d/apache-logrotate on MTB-WEB01 and the System Log Forwarder unit on MTB-FILE01
mtb_detect_02	A file with a script extension written into a directory that the web server serves and that accepts uploads	style.php appearing in /var/www/html/uploads/ at 02:15:22
mtb_detect_03	A web request whose path contains encoded directory traversal against a CGI path	The /cgi-bin/.%2e/.%2e/ requests on 30 August and 2 September
mtb_detect_04	An SSH login accepted for a service account originating from a server-segment host rather than a workstation	svc-backup accepted from 10.50.2.24 on MTB-FILE01 at 02:52:14

10.6 Requirements for every rule you write

1. Include all four mandatory fields, plus description, tags, falsepositives and level. A rule with no falsepositives entry will not be accepted.
2. Tag it with at least one ATT&CK technique from your Section 9 mapping, in the form **attack.t1053.003**. Lower case, dots as separators.
3. Write the description so it says what the rule detects and why it exists. Reference the case number.
4. Set the level honestly. Not everything is critical. A rule that fires on something needing review this week is medium, not critical.
5. In the falsepositives field, name at least one specific legitimate activity that would match. If you cannot think of one, your rule is probably too broad and you have not thought about it hard enough.
6. Below each rule, in your report, write two sentences: which Pyramid tier it targets, and what the attacker would have to change to evade it.

The most common mistake on this section

Writing a rule that matches 185.220.101.47. It will detect this incident perfectly and nothing else ever again.

Before you save each rule, ask: if the attacker came back tomorrow from a different address, with different filenames, would this still fire? If the answer is no, rewrite it.

That question is the entire discipline of detection engineering.

mtb_detect_01

title: Persistence Entry Executing Binary From /usr/local

id: 3a7f9c12-6e4d-4b8a-9f21-7c5d8e2a1b09

status: experimental

description: |

Detects a cron or systemd persistence entry that executes a binary located under /usr/local/. Attackers commonly stage payloads here because it looks like a legitimate local install path, rather than a distro-managed location.

Observed on MTB-WEB01 (/etc/cron.d/apache-logrotate) and MTB-FILE01 (systemd "Log Forwarder" unit) during case MTB-IR-2026-0902.

references:

- <https://attack.mitre.org/techniques/T1053/003/>
- <https://attack.mitre.org/techniques/T1543/002/>

author: Meridian Trust Bank SOC

date: 2026-09-04

tags:

- attack.t1053.003
- attack.t1543.002

logsource:

product: linux

service: file_event

detection:

keywords:

- '/usr/local/bin/'
- '/usr/local/sbin/'

condition: keywords

falsepositives:

- Legitimate in-house tooling or monitoring agents deployed to /usr/local/ and scheduled via cron or systemd as part of an approved change.

level: high

NOTE: Targets Tier 4 (host artifacts) on the Pyramid of Pain. The attacker evades this by relocating the payload out of /usr/local/ to any other writable path (/opt/, /var/tmp/, a user home directory).

mtb_detect_02

title: Script File Written Into Web-Servable Upload Directory

id: 8d1e4f76-2b9a-4c5e-a8f0-1d6c3e9b5a72

status: experimental

description: |

Detects creation of a file with an executable script extension inside a directory the web server serves publicly and that accepts uploads. A script file should never legitimately appear in an uploads-only path; its presence is a strong web shell indicator. Observed as style.php in /var/www/html/uploads/ at 02:15:22 during case MTB-IR-2026-0902.

references:

- <https://attack.mitre.org/techniques/T1505/003/>

author: Meridian Trust Bank SOC

date: 2026-09-04

tags:

- attack.t1505.003

logsource:

product: linux

service: file_event

detection:

path_selector:

TargetFilename|contains: '/uploads/'

extension_selector:

TargetFilename|endswith:

- '.php'
- '.phtml'
- '.php5'
- '.jsp'
- '.asp'

- '.aspx'

condition: path_selector and extension_selector

falsepositives:

- A CMS or plugin update process that legitimately writes script files into an uploads directory as part of a documented, scheduled deployment.

level: critical

NOTE: Targets Tier 4 (host artifacts) on the Pyramid of Pain. The attacker evades this by naming the payload with an extension not covered by the detection list (e.g. .php7, .phar, or a server-specific handler mapping), or by dropping it outside the /uploads/ path if any other writable, web-servable directory exists.

mtb_detect_03

title: Encoded Directory Traversal Against CGI Path

id: f4b6a983-1c7e-45d2-9a3b-6e0f2d8c4b15

status: experimental

description: |

Detects a web request whose path contains URL-encoded directory traversal sequences targeted against a CGI endpoint, consistent with exploitation of the Apache HTTP Server path traversal and RCE flaw (CVE-2021-41773 / CVE-2021-42013). Observed against MTB-WEB01's /cgi-bin/ path on both 30 August and 2 September during case MTB-IR-2026-0902.

references:

- <https://attack.mitre.org/techniques/T1190/>

author: Meridian Trust Bank SOC

date: 2026-09-04

tags:

- attack.t1190

logsource:

category: webserver

product: apache

detection:

path_selector:

cs-uri-stem|contains: '/cgi-bin/'

traversal_selector:

cs-uri-stem|contains: '%2e%2e'

condition: path_selector and traversal_selector

falsepositives:

- An authorized internal vulnerability scan or penetration test targeting this same CGI path, if not excluded by source IP or a scheduled test window.

level: high

NOTE: Targets Tier 6 (TTPs), the top of the pyramid, since this matches the exploitation technique itself rather than any single file, hash, or IP. To evade it the attacker has to switch to a materially different traversal encoding (double encoding, Unicode variants) or abandon the CGI-path technique entirely for a different vulnerability class.

mtb_detect_04

title: Service Account SSH Login Originating From Server Segment

id: 6c2e8a45-9f1b-4d7c-b3e6-0a5d7f9c2e83

status: experimental

description: |

Detects an SSH login accepted for a designated service account where the source address belongs to the server segment rather than an expected workstation or admin-jump host. Legitimate service-account logins should originate from known management sources, not from another server. Observed as svc-backup accepted from 10.50.2.24 (MTB-WEB01) on MTB-FILE01 at 02:52:14 during case MTB-IR-2026-0902.

references:

- <https://attack.mitre.org/techniques/T1021/004/>
- <https://attack.mitre.org/techniques/T1078/003/>

author: Meridian Trust Bank SOC

date: 2026-09-04

tags:

- attack.t1021.004
- attack.t1078.003

logsource:

product: linux

service: auth

detection:

keywords:

- 'Accepted publickey for svc-backup'
- 'Accepted publickey for svc-deploy'
- 'Accepted publickey for svc-monitor'

source_segment:

src_ip|cidr: '10.50.2.0/24'

condition: keywords and source_segment

falsepositives:

- A documented service-to-service integration where one server segment host is an approved source for a given service account (e.g. an orchestration host managing the fleet) — verify against the current access list before tuning this out.

level: medium

NOTE: Targets Tier 6 (TTPs), since it's catching the behavioral pattern of lateral movement via a valid account from an unexpected network position, not a specific IOC. To evade it the attacker would need to either compromise a host that's already an approved source for that service account, or avoid server-to-server SSH entirely in favor of a different lateral-movement path.

10.7 Record: your detection set

Rule file	What it detects	ATT&CK tag	Level	Pyramid tier
mtb_detect_01	A cron or systemd persistence entry that executes a binary from /usr/local/	attack.t1053.003, attack.t1543.002	High	4
mtb_detect_02	A script file with an executable extension written into a web-served uploads directory	attack.t1505.003	Critical	4
mtb_detect_03	A web request with encoded directory traversal against a CGI path	attack.t1190	High	6
mtb_detect_04	An SSH login accepted for a service account from a server-segment host rather than an expected workstation/admin source	attack.t1021.004, attack.t1078.003	Medium	6

11. Day 5, Part 3: Containment, Eradication and Recovery

Two hosts are compromised and you know how. The bank now needs them back in service, safely, and it needs to be certain the attacker is gone rather than hoping so.

11.1 The three phases, and why the order matters

Phase	Goal	Gets this wrong how
Containment	Stop the attacker acting, without destroying evidence or tipping them off before you are ready.	Rebuilding a host immediately, which removes the attacker and every trace of what they did.
Eradication	Remove every access path they established, not just the one you found first.	Deleting the webshell and declaring victory while three other persistence mechanisms remain.
Recovery	Return to normal service with confidence, and watch for their return.	Restoring from a backup taken after the compromise, which restores the compromise.

The backup question you must answer in your plan

The intrusion began on 28 August. Any backup of MTB-WEB01 taken after the first successful exploitation may contain attacker artefacts.

Your recovery plan has to state which backup generation is safe to restore from, and how you would verify that before trusting it.

Teams that skip this restore the incident along with the service and then spend a fortnight wondering how the attacker got back in.

11.2 Build the containment plan

1. List every containment action for both compromised hosts, in the order you would perform them. For each, state what it stops and what evidence it risks destroying.

Order	Action	Host	What it stops	Evidence it risks destroying
1	Network-isolate via ACL	MTB-WEB01	Any further C2 traffic, further exfiltration, any further lateral movement from this host	None
2	Network-isolate via ACL	MTB-FI LE01	Closes the beacon to 185.220.102.244:443 and stops further exfiltration of file-server contents	Minimal to none
3	Capture volatile state	MTB-FI LE01	Nothing	None
4	Disk image acquisition	MTB-FI LE01	Nothing	None

5	Revoke/disable the attacker's SSH key and the svc-backup account only after imaging both hosts	BOTH	Any renewed access via the known credential	None
---	--	------	---	------

- Decide whether to power the hosts off, isolate them at the network, or leave them running under observation. Justify the choice using what you learned about order of volatility on Day 3.

Isolate at the network layer, we do not want to risk losing volatile evidence by powering off machines.

- Identify what containment must happen on hosts that are not compromised. The attacker had a private key; the exposure is not limited to the two hosts they used it on.
 - Audit authorized_keys on every host in the fleet for the attacker's specific public key, and for any unexpected svc-backup or similarly-named account.
 - Rotate keys so the private key is worthless

11.3 Build the eradication plan

List every access path the attacker established. There are more than most people count on the first attempt. Work through both hosts systematically.

Access path	Host(s)	How you remove it	How you verify removal
Root SSH key in /root/.ssh/authorized_keys	MTB-WEB01	Delete the attacker's key line; leave only verified legitimate keys	Diff the file's contents against a known-good baseline; confirm the attacker's ed25519 key (...ATTACKER00000000 attacker@blackfin) is absent; attempt an SSH login using the attacker's public key and confirm it's refused
svc-backup account, UID=0	MTB-WEB01	userdel -r svc-backup; also check for and remove any sudoers/group entries added for it	grep svc-backup /etc/passwd /etc/group /etc/shadow returns nothing; confirm no UID=0 entries remain besides root
SSH key in /home/svc-backup/.ssh/authorized_keys	MTB-WEB01	Removed automatically once the account is deleted (-r flag)	Confirm /home/svc-backup/ no longer exists
svc-backup account, UID=0	MTB-FILE01	userdel -r svc-backup on FILE01 independently	grep svc-backup /etc/passwd /etc/group /etc/shadow on FILE01 returns nothing
Shared /home mount trusting a single planted key across hosts	MTB-WEB01, MTB-FILE01	Either segment the mount so authorized_keys isn't shared fleet-wide, or move to per-host key management	Enumerate every host with the shared mount; confirm none of them still honor the attacker's public key

Access path	Host(s)	How you remove it	How you verify removal
Web shell /var/www/html/uploads/style.php	MTB-WEB01	Delete the file	File absent; find /var/www/html -newer <known-clean-baseline> -type f returns nothing unexpected
Cron persistence /etc/cron.d/apache-logrotate	MTB-WEB01	Delete the file; restart cron	File absent
Payload/beacon binary /usr/local/bin/apache-worker	MTB-WEB01	Delete the binary; kill the running process	`ps -ef`
Systemd persistence unit ("Log Forwarder")	MTB-FILE01	systemctl stop, systemctl disable, then delete the unit file from /etc/systemd/system/	systemctl list-units no longer shows the unit
Payload/beacon binary /usr/local/sbin/syslog-forward	MTB-FILE01	Delete the binary; kill the running process	`ps -ef`
Staged exfil archive /tmp/.ICE-unix/x and the directory itself	MTB-WEB01	Delete the archive and the hidden staging directory	Directory absent from /tmp/
Unpatched path-traversal vulnerability (CVE-2021-42013) + permissive <Directory />ScriptAlias config	MTB-WEB01	Upgrade Apache to $\geq 2.4.51$; correct the <Directory /> block to Require all denied with explicit scoping to /var/www/html; remove or tightly scope ScriptAlias /cgi-bin/	apache2 -v shows $\geq 2.4.51$
Unpatched pkexec/polkit vulnerability (CVE-2021-4034 / PwnKit)	MTB-WEB01, MTB-FILE01	Upgrade policykit-1 to the patched build for the distro release	apt changelog policykit-1 lists CVE-2021-4034 as fixed in the installed version

1. Complete the table. Include everything: accounts, keys, scheduled execution, services, dropped files, and the vulnerabilities that let them in.
2. Identify every credential and key that must be treated as compromised, and state the blast radius of each. For the SSH private key, that means every host that trusted it, not just the two you found it used on.
3. State how you would confirm eradication is complete. Your Section 10 detection rules are part of the answer.

11.4 Build the recovery plan

1. State the order in which the two hosts return to service, and why that order.

MTB-FILE01 returns to service before MTB-WEB01.

The reasoning is about exposure, not severity: MTB-WEB01 is the DMZ, internet-facing host, its whole reason for existing is to be reachable from the outside, which means it's the host most exposed to the vulnerability that started this entire incident (CVE-2021-42013, a remotely exploitable flaw). MTB-FILE01 is an internal fleet file server, reached only via internal lateral movement, not directly from the internet. Bringing FILE01 back first lets you restore internal file services with a much smaller blast radius if something was missed, while WEB01 stays offline until you can be fully confident the internet-facing attack surface itself is closed. Reintroducing the DMZ host too early could re-open the exact door the attacker used.

2. State which backup generation is safe, based on your revised start of intrusion, and how you would verify it before restoring.

The revised start of intrusion is 2026-08-28 01:52:17 (first contact from 185.220.101.47, egress log). Any backup taken on or after August 28 is potentially within the compromise window and cannot be trusted as clean. The safe backup generation is the most recent one taken before August 28. Realistically, this likely means going back further than initially assumed, since even an August 27 backup would only be one day removed from first contact and worth extra scrutiny given the attacker's five-day dwell time before any exploitation activity began.

Verification before restoring from it: don't just trust the date stamp, actively check the backup itself against every indicator this investigation has produced. No svc-backup account in /etc/passwd, no attacker key in any authorized_keys file, no /etc/cron.d/apache-logrotate, no apache-worker/syslog-forward binaries, Apache still at the pre-attack version, and file hashes for anything restorable compared against the SHA-256 values already on record.

3. State what must be true before MTB-WEB01 is exposed to the internet again. Both vulnerabilities from your Day 1-2 assessment are part of this, and so is the Apache configuration.
 - CVE-2021-42013 patched. Apache upgraded to $\geq 2.4.51$, confirmed via `apache2 -v`, not just a package-manager claim.
 - CVE-2021-4034 (PwnKit) patched. `policykit-1` upgraded to the security-fixed build, confirmed via the package changelog naming the CVE as resolved.
 - Apache configuration corrected. The `<Directory />` block set back to `Require all denied` with access explicitly scoped to `/var/www/html`, and `ScriptAlias /cgi-bin/` either removed or tightly restricted, per the Day 1-2 finding (MTB-0001) that named this configuration as what made the version-level flaw actually exploitable. Patching the version without fixing this config leaves the host still restorable to the same failure mode by any other future vulnerability that touches the filesystem.
 - Every item in the Section 11.3 eradication table verified clean on this specific host (accounts, keys, cron entries, dropped files).
 - Host rebuilt or restored from the verified-clean backup generation, not patched in place on top of a potentially still-compromised filesystem. Patching known vulnerabilities doesn't remove unknown backdoors that might still be sitting there.
4. Define a heightened monitoring period: how long, watching for what, and using which of your four detection rules.

Duration: minimum 30 days from return to service

What to watch for:

- Any outbound connection to 185.220.101.47, 185.220.102.244, or any new address in the same address ranges
- Recurrence of svc-backup, or any new `UID=0` account outside a documented break-glass change
- Any file written to `/etc/cron.d/` or a new systemd unit executing a binary from `/usr/local/`
- Any script file appearing under `/var/www/html/uploads/`
- Repeat scanning/traversal patterns against `/cgi-bin/`

Which detection rules:

All four. mtb_detect_01 covers the cron/systemd persistence recurrence, mtb_detect_02 covers a repeat web shell drop, mtb_detect_03 covers the exact initial-access technique reoccurring, and mtb_detect_04 covers any lateral-movement attempt via a service account from within the server segment.

Attackers come back

They know the environment, they know what worked, and they often still hold a credential nobody thought to rotate. A recovery plan that ends the moment service is restored is only half a plan. Say explicitly what is watched, for how long, and who is watching it.

12. Day 5, Part 4: Post-Incident Review

The last deliverable is the one that decides whether any of this work matters beyond this incident. A post-incident review asks what let this happen and what changes so that it does not happen the same way twice.

12.1 The detection gap

Work this out precisely, because it is the core finding of the review.

Moment	Timestamp	How long after first activity
First attacker activity	2026-08-28 01:52:17 UTC	Baseline
First successful exploitation	2026-09-02 02:12:33 UTC	5 days, 20 min, 16 sec
Second host compromised	2026-09-02 02:52:14 UTC	5 days, 59 min, 57 sec
Largest data transfer	2026-09-02 05:12:19 UTC	5 days, 3 hr, 20 min, 2 sec
Alert reached a human	2026-09-02 09:47:12 UTC	5 days, 7 hr, 54 min, 55 sec

1. Fill in the table from your extended timeline.
2. For each of your four detection rules from Section 10, state when it would have fired. Then state how much of the intrusion would have been prevented if the SOC had acted on that first firing.

mtb_detect_01

Fires at: 2026-09-02 02:26:30 UTC

Prevention: Beacon establishment (02:31:12), the exfil staging (02:34:55), the 2.88 MB transfer (02:38:02), and the lateral move to FILE01 (02:52:14).

mtb_detect_02

Fires at: 2026-09-02 02:15:22 UTC

Prevention: pkexec privesc, svc-backup creation, both SSH backdoors, cron persistence, the beacon, the lateral move to FILE01, and both data transfers.

mtb_detect_03

Fires at: 2026-09-02 02:12:33 UTC

Prevention: the web shell drop, the pkexec privilege escalation, the svc-backup account creation on both hosts, both SSH backdoors, both persistence mechanisms (cron on WEB01, systemd on FILE01), both beacon processes, the lateral move to FILE01, and both data transfers.

mtb_detect_04

Fires at: 2026-09-02 02:52:14 UTC

Prevention: The account creation, systemd persistence, FILE01 beacon, and the 9.92 MB transfer on FILE01.

3. Identify which single rule would have made the biggest difference, and say why.

mtb_detect_03 would have made the biggest difference because it's the earliest point in the entire kill chain where the attack could have been stopped before the attacker gained any foothold on the host.

4. Note honestly what none of your rules would have caught. Every detection set has gaps, and a review that claims otherwise is not credible.
 - The five-day reconnaissance and scanning window itself (2026-08-28 through 08-30, T1595.002). None of these four rules watch for scan/probe behavior; every one of them only triggers once exploitation is already underway or succeeding. The attacker's entire initial presence on the network was invisible to this detection set.
 - The privilege escalation event itself (T1068, pkexec/PwnKit at 02:20:47). No rule here detects the exploit directly. `mtb_detect_01` only catches the persistence artifact the attacker built afterward, not the escalation that gave them root in the first place.
 - The rogue UID=0 account creation itself (T1136.001). This is a real, specific gap: nothing in this four-rule set watches `/etc/passwd` or `useradd` output for a duplicate UID 0, even though that exact detection exists elsewhere in this case and isn't part of what's being scored here.
 - Anti-forensic activity (T1070.003, the `.bash_history` clearing on both root and `mtbadmin`)
 - Exfiltration as a volume anomaly. None of these rules look at transfer size; they only catch the mechanism delivering it (persistence, web shell, lateral SSH), so a transfer riding a channel these rules don't cover would go completely undetected.
 - Straightforward evasion of every single rule as written. Relocate the payload path out of `/usr/local/` and `mtb_detect_01` never fires; use an uncommon script extension and `mtb_detect_02` never fires; switch traversal encoding (double-encoding, Unicode) and `mtb_detect_03` never fires; move laterally from a host already on the approved-source list for that service account and `mtb_detect_04` never fires. None of these evasions require new capability from the attacker, just minor variations on what they already did here.

12.2 The uncomfortable questions

A post-incident review that only discusses the attacker has failed. Answer all four of these, plainly.

- **Why was a host with a publicly known, actively exploited vulnerability still exposed to the internet in September,** when the assessment on 8 September flagged it and it had been in the KEV catalogue since 2021? Your answer must address the patching process, not the attacker.

The KEV finding sat unpatched for years after public disclosure, this was purely a patch management failure.

- **Why did the assessment run on 8 September, after the first successful exploitation on 30 August?** What scan cadence would have caught this in time, and what would that cost?

The scan cadence was periodic, not continuous, so the gap between scans on this internet-facing host happened to contain the attack. Daily external scanning on DMZ assets would have closed that gap, at the cost of tooling and daily triage time for a small host count.

- **Why was an unattended SSH private key trusted by other hosts sitting on an internet-facing server in the DMZ?** What should the trust relationship between the DMZ and the server segment look like?

The DMZ and the internal server segment shared filesystem-level trust (a shared `/home` mount), so a key planted on the internet-facing host worked internally with no extra exploit needed. The DMZ should trust nothing from the internal segment, and nothing internal should trust anything from the DMZ.

- **Why did seven and a half hours of five-minute beaconing to a non-allowlisted destination not alert anyone sooner?** The egress log recorded every one of those connections. The data existed. What was missing?

The alert logic waited to confirm a repeating pattern before firing, even though the data existed from the first connection. What's missing is alerting on the first contact with a non-allowlisted destination, not waiting for periodicity to prove itself.

Write this without blaming a person

Every one of those questions has an answer that is about a process, a control or a priority. None of them has a useful answer that is about an individual being careless.

Reviews that name individuals stop people reporting things. Within a year the organisation is less safe than before, because the next person who notices something odd decides not to mention it.

Write about what the system allowed, not about who did it.

12.3 Report structure to follow

Section	Contents
Executive summary	Three to five sentences for a non-technical reader. What happened, what was lost, what changes.
What happened	The full intrusion in plain language, from first activity to eradication. No CVE numbers here.
ATT&CK mapping	Your completed table from Section 9.
Detection gap	The timeline table from Section 12.1 and what each rule would have changed.
New detections	Your four Sigma rules, with the Pyramid tier and evasion note for each.
Containment, eradication, recovery	Your plans from Section 11, including the backup decision.
Root cause and contributing factors	Your answers to Section 12.2. Process and control failures, not people.
Recommendations	Specific, owned and dated. Each one traceable to something in this review.

Executive Summary

Between late August and early September 2026, an external attacker broke into Meridian Trust Bank's public-facing web server by exploiting a known software weakness that had gone unpatched for years, then used that access to plant hidden backdoors, create a fake administrator account, and move to a second internal server. The attacker had a presence on the network for roughly five days before the break-in that led to today's incident, and remained undetected on the systems for nearly eight hours after that, during which time nearly 13 megabytes of data, including web server files and content from the company's file server, was sent to the attacker's own infrastructure. Both affected servers have been isolated and are being rebuilt from clean, verified backups. No evidence currently confirms exactly what data left the network, and closing that gap is now the top open item. This incident happened because a known, publicly documented vulnerability was left unpatched well past when it should have been fixed, and it is driving concrete changes to patching timelines, network trust between systems, and how quickly the security team is alerted to suspicious activity going forward.

What Happened

The attacker first made contact with the bank's web server five days before the actual break-in, quietly testing the door before walking through it. On the day of the intrusion, they found and exploited a long-known weakness in the web server software to plant a hidden tool (disguised as a normal file) that gave them the ability to run commands on the server. Within about six minutes of that first foothold, they used a second, unrelated weakness on the same machine to escalate themselves from a low-level web account to full administrator control.

Once they had that level of access, the attacker moved fast and covered their tracks carefully. They created a fake maintenance account designed to blend in, gave themselves a hidden key that let them log back in at will, and set up a scheduled task disguised as routine housekeeping to keep their access alive even if their initial tool was found and removed. They then packaged up a copy of website data and sent nearly 3 megabytes of it out to their own servers. About 31 minutes after gaining administrator control, they used the hidden key they'd planted to quietly walk into a second internal server, the company's file server, using a trust relationship between the two systems that should not

have existed. On that second machine, they repeated the same playbook: a fake account, a hidden backdoor disguised as a legitimate background service, and roughly three hours later, a much larger transfer of nearly 10 megabytes of file server data sent to the attacker.

The attacker also attempted to erase evidence of their activity by clearing command history on two accounts, though the attempt was only partially successful on one of them, which is part of how this activity was later reconstructed.

The intrusion was ultimately flagged by automated monitoring almost eight hours after the initial break-in, when the pattern of the attacker's ongoing hidden connection became consistent enough to trigger an alert. The affected web server was disconnected from the network within about 20 minutes of that alert. The investigation that followed identified the full chain of events described above, confirmed a second compromised system that the original alert never flagged, and produced a plan to remove every piece of attacker access, fix the underlying weaknesses that allowed this to happen, and restore both systems safely

ATT&CK Mapping

Tactic	Technique ID	Technique name	Your evidence
Reconnaissance	T1595.002	Vulnerability Scanning	Egress log, 28 Aug 01:52, 214 sessions to port 80 carrying zero bytes
Initial Access	T1190	Exploit Public-Facing Application	Access log, 02:12:33, GET /cgi-bin/%2e%2e/%2e%2e/etc/passwd → 200/622 bytes (curl/7.81.0). Matches CVE-2021-42013 against Apache 2.4.49-1ubuntu4 (Finding MTB-0001)
Persistence	T1505.003	Web Shell	Filesystem timeline, /var/www/html/uploads/style.php created 02:15:22 via traversal RCE POST; recovered file content confirms PHP web shell disguised as a stylesheet
Privilege Escalation	T1068	Exploitation for Privilege Escalation	Auth log, 02:20:47–02:21:05, pkexec[1207] opens a root session on behalf of www-data (uid 33), executes /bin/sh as root from CWD=/var/www/html/uploads. Consistent with CVE-2021-4034 (PwnKit)
Persistence	T1136.001	Local Account	Auth log, 02:23:11 WEB01: useradd svc-backup, UID=0, GID=0; repeated on FILE01, 02:54:39, identical UID=0 warning signature
Persistence	T1098.004	SSH Authorized Keys	Filesystem timeline, /root/.ssh/authorized_keys written 02:24:48; /home/svc-backup/.ssh/authorized_keys written 02:24:50; recovered content shows attacker key commented attacker@blackfin
Persistence	T1053.003	Cron	Filesystem timeline + system log, /etc/cron.d/apache-logrotate written 02:26:30; cron[812]: RELOAD logged 02:26:31

Tactic	Technique ID	Technique name	Your evidence
Persistence	T1543.002	Systemd Service	FILE01 auth log, systemd[1]: Reloading / Started Log Forwarder at 02:56:02–03; process telemetry shows /usr/local/sbin/syslog-forward launched by systemd, parent - (daemon)
Credential Access	T1552.004	Private Keys	No direct evidence found
Lateral Movement	T1021.004	SSH	FILE01 auth log, 02:52:14, Accepted publickey for svc-backup from 10.50.2.24 port 52118. WEB01's own IP, using the key planted at 02:24:50
Collection	T1560.001	Archive via Utility	Root .bash_history: tar czf /tmp/.ICE-unix/x /var/www/html/data; filesystem timeline confirms /tmp/.ICE-unix created 02:34:55
Command and Control	T1071.001	Web Protocols	Volatile capture + process telemetry: apache-worker (WEB01) ESTABLISHED to 185.220.101.47:8443; syslog-forward (FILE01) ESTABLISHED to 185.220.102.244:443
Exfiltration	T1041	Exfiltration Over C2 Channel	Egress log: WEB01, 02:38:02, 2,884,416 bytes to 185.220.101.47:8443 (same port/destination as the beacon); FILE01, 05:12:19, 9,917,722 bytes to 185.220.102.244:443 (same port/destination as its beacon)
Stealth	T1070.003	Clear Command History	Filesystem timeline, root .bash_history touched 02:41:19 (history -c recovered in content, only partially effective; 465 bytes remained); mtbadmin/.bash_history wiped to 0 bytes at 02:41:22
Discovery	T1082	System Information Discovery	Root .bash_history (recovered file, 11.4): id, uname -a, cat /etc/os-release, which curl wget python3. The attacker's first actions after gaining the root shell, checking the OS and available tooling before building persistence
Persistence / Lateral Movement	T1078.003	Valid Accounts: Local Accounts	WEB01 auth log, 02:24:48, root logs in via SSH using the planted key; no exploit required, just a credential; FILE01 auth log, 02:52:14, svc-backup authenticates via SSH the same way. Both are the attacker using an account as a legitimate credential rather than exploiting anything

Detection Gap

Moment	Timestamp	How long after first activity
First attacker activity	2026-08-28 01:52:17 UTC	Baseline
First successful exploitation	2026-09-02 02:12:33 UTC	5 days, 20 min, 16 sec
Second host compromised	2026-09-02 02:52:14 UTC	5 days, 59 min, 57 sec
Largest data transfer	2026-09-02 05:12:19 UTC	5 days, 3 hr, 20 min, 2 sec
Alert reached a human	2026-09-02 09:47:12 UTC	5 days, 7 hr, 54 min, 55 sec

New Detection mtb_detect_01

title: Persistence Entry Executing Binary From /usr/local

id: 3a7f9c12-6e4d-4b8a-9f21-7c5d8e2a1b09

status: experimental

description: |

Detects a cron or systemd persistence entry that executes a binary located under /usr/local/. Attackers commonly stage payloads here because it looks like a legitimate local install path, rather than a distro-managed location.

Observed on MTB-WEB01 (/etc/cron.d/apache-logrotate) and MTB-FILE01 (systemd "Log Forwarder" unit) during case MTB-IR-2026-0902.

references:

- <https://attack.mitre.org/techniques/T1053/003/>
- <https://attack.mitre.org/techniques/T1543/002/>

author: Meridian Trust Bank SOC

date: 2026-09-04

tags:

- attack.t1053.003
- attack.t1543.002

logsource:

product: linux

service: file_event

detection:

keywords:

- '/usr/local/bin/'
- '/usr/local/sbin/'

condition: keywords

falsepositives:

- Legitimate in-house tooling or monitoring agents deployed to /usr/local/ and scheduled via cron or systemd as part of an approved change.

level: high

NOTE: Targets Tier 4 (host artifacts) on the Pyramid of Pain. The attacker evades this by relocating the payload out of /usr/local/ to any other writable path (/opt/, /var/tmp/, a user home directory).

mtb_detect_02

title: Script File Written Into Web-Servable Upload Directory

id: 8d1e4f76-2b9a-4c5e-a8f0-1d6c3e9b5a72

status: experimental

description: |

Detects creation of a file with an executable script extension inside a directory the web server serves publicly and that accepts uploads. A script file should never legitimately appear in an uploads-only path; its presence is a strong web shell indicator. Observed as style.php in /var/www/html/uploads/ at 02:15:22 during case MTB-IR-2026-0902.

references:

- <https://attack.mitre.org/techniques/T1505/003/>

author: Meridian Trust Bank SOC

date: 2026-09-04

tags:

- attack.t1505.003

logsource:

product: linux

service: file_event

detection:

path_selector:

TargetFilename|contains: '/uploads/'

extension_selector:

TargetFilename|endswith:

- '.php'
- '.phtml'
- '.php5'
- '.jsp'
- '.asp'
- '.aspx'

condition: path_selector and extension_selector

falsepositives:

- A CMS or plugin update process that legitimately writes script files into an uploads directory as part of a documented, scheduled deployment.

level: critical

NOTE: Targets Tier 4 (host artifacts) on the Pyramid of Pain. The attacker evades this by naming the payload with an extension not covered by the detection list (e.g. .php7, .phar, or a server-specific handler mapping), or by dropping it outside the /uploads/ path if any other writable, web-servable directory exists.

mtb_detect_03

title: Encoded Directory Traversal Against CGI Path

id: f4b6a983-1c7e-45d2-9a3b-6e0f2d8c4b15

status: experimental

description: |

Detects a web request whose path contains URL-encoded directory traversal sequences targeted against a CGI endpoint, consistent with exploitation of the Apache HTTP Server path traversal and RCE flaw (CVE-2021-41773 / CVE-2021-42013). Observed against MTB-WEB01's /cgi-bin/ path on both 30 August and 2 September during case MTB-IR-2026-0902.

references:

- <https://attack.mitre.org/techniques/T1190/>

author: Meridian Trust Bank SOC

date: 2026-09-04

tags:

- attack.t1190

logsource:

category: webserver

product: apache

detection:

path_selector:

cs-uri-stem|contains: '/cgi-bin/'

traversal_selector:

cs-uri-stem|contains: '%2e%2e'

condition: path_selector and traversal_selector

falsepositives:

- An authorized internal vulnerability scan or penetration test targeting this same CGI path, if not excluded by source IP or a scheduled test window.

level: high

NOTE: Targets Tier 6 (TTPs), the top of the pyramid, since this matches the exploitation technique itself rather than any single file, hash, or IP. To evade it the attacker has to switch to a materially different traversal encoding (double encoding, Unicode variants) or abandon the CGI-path technique entirely for a different vulnerability class.

mtb_detect_04

title: Service Account SSH Login Originating From Server Segment

id: 6c2e8a45-9f1b-4d7c-b3e6-0a5d7f9c2e83

status: experimental

description: |

Detects an SSH login accepted for a designated service account where the source address belongs to the server segment rather than an expected workstation or admin-jump host. Legitimate service-account logins should originate from known management sources, not from another server. Observed

as svc-backup accepted from 10.50.2.24 (MTB-WEB01) on MTB-FILE01 at 02:52:14 during case MTB-IR-2026-0902.

references:

- <https://attack.mitre.org/techniques/T1021/004/>
- <https://attack.mitre.org/techniques/T1078/003/>

author: Meridian Trust Bank SOC

date: 2026-09-04

tags:

- attack.t1021.004
- attack.t1078.003

logsource:

product: linux

service: auth

detection:

keywords:

- 'Accepted publickey for svc-backup'
- 'Accepted publickey for svc-deploy'
- 'Accepted publickey for svc-monitor'

source_segment:

src_ip|cidr: '10.50.2.0/24'

condition: keywords and source_segment

falsepositives:

- A documented service-to-service integration where one server segment host is an approved source for a given service account (e.g. an orchestration host managing the fleet) — verify against the current access list before tuning this out.

level: medium

NOTE: Targets Tier 6 (TTPs), since it's catching the behavioral pattern of lateral movement via a valid account from an unexpected network position, not a specific IOC. To evade it the attacker would need to either compromise a host that's already an approved source for that service account, or avoid server-to-server SSH entirely in favor of a different lateral-movement path.

Containment, Eradication, Recover

Containment

Order	Action	Host	What is stops	Evidence it risks destroying
1	Network-isolate via ACL	MTB-WEB01	Any further C2 traffic, further exfiltration, any further lateral movement from this host	None
2	Network-isolate via ACL	MTB-FI LE01	Closes the beacon to 185.220.102.244:443 and stops further exfiltration of file-server contents	Minimal to none
3	Capture volatile state	MTB-FI LE01	Nothing	None
4	Disk image acquisition	MTB-FI LE01	Nothing	None
5	Revoke/disable the attacker's SSH key and the svc-backup account only after imaging both hosts	BOTH	Any renewed access via the known credential	None

- Decide whether to power the hosts off, isolate them at the network, or leave them running under observation. Justify the choice using what you learned about order of volatility on Day 3.

Isolate at the network layer, we do not want to risk losing volatile evidence by powering off machines.

- Identify what containment must happen on hosts that are not compromised. The attacker had a private key; the exposure is not limited to the two hosts they used it on.
 - Audit `authorized_keys` on every host in the fleet for the attacker's specific public key, and for any unexpected `svc-backup` or similarly-named account.
 - Rotate keys so the private key is worthless

Eradication

Access path	Host(s)	How you remove it	How you verify removal
Root SSH key in <code>/root/.ssh/authorized_keys</code>	MTB-WEB01	Delete the attacker's key line; leave only verified legitimate keys	Diff the file's contents against a known-good baseline; confirm the attacker's <code>ed25519</code> key (<code>...ATTACKER00000000 attacker@blackfin</code>) is absent; attempt an SSH login using the attacker's public key and confirm it's refused

Access path	Host(s)	How you remove it	How you verify removal
svc-backup account, UID=0	MTB-WEB01	userdel -r svc-backup; also check for and remove any sudoers/group entries added for it	grep svc-backup /etc/passwd /etc/group /etc/shadow returns nothing; confirm no UID=0 entries remain besides root
SSH key in /home/svc-backup/.ssh/authorized_keys	MTB-WEB01	Removed automatically once the account is deleted (-r flag)	Confirm /home/svc-backup/ no longer exists
svc-backup account, UID=0	MTB-FILE01	userdel -r svc-backup on FILE01 independently	grep svc-backup /etc/passwd /etc/group /etc/shadow on FILE01 returns nothing
Shared /home mount trusting a single planted key across hosts	MTB-WEB01, MTB-FILE01	Either segment the mount so authorized_keys isn't shared fleet-wide, or move to per-host key management	Enumerate every host with the shared mount; confirm none of them still honor the attacker's public key
Web shell /var/www/html/uploads/style.php	MTB-WEB01	Delete the file	File absent; find /var/www/html -newer <known-clean-baseline> -type f returns nothing unexpected
Cron persistence /etc/cron.d/apache-logrotate	MTB-WEB01	Delete the file; restart cron	File absent
Payload/beacon binary /usr/local/bin/apache-worker	MTB-WEB01	Delete the binary; kill the running process	`ps -ef`
Systemd persistence unit ("Log Forwarder")	MTB-FILE01	systemctl stop, systemctl disable, then delete the unit file from /etc/systemd/system/	systemctl list-units no longer shows the unit
Payload/beacon binary /usr/local/sbin/syslog-forward	MTB-FILE01	Delete the binary; kill the running process	`ps -ef`
Staged exfil archive /tmp/.ICE-unix/x and the directory itself	MTB-WEB01	Delete the archive and the hidden staging directory	Directory absent from /tmp/
Unpatched path-traversal vulnerability (CVE-2021-42013) + permissive <Directory />ScriptAlias config	MTB-WEB01	Upgrade Apache to $\geq 2.4.51$; correct the <Directory /> block to Require all denied with explicit scoping to /var/www/html; remove or tightly scope ScriptAlias /cgi-bin/	apache2 -v shows $\geq 2.4.51$

Access path	Host(s)	How you remove it	How you verify removal
Unpatched pkexec/polkit vulnerability (CVE-2021-4034 / PwnKit)	MTB-WEB01, MTB-FILE01	Upgrade policykit-1 to the patched build for the distro release	apt changelog policykit-1 lists CVE-2021-4034 as fixed in the installed version

Recovery

5. State the order in which the two hosts return to service, and why that order.

MTB-FILE01 returns to service before MTB-WEB01.

The reasoning is about exposure, not severity: MTB-WEB01 is the DMZ, internet-facing host, its whole reason for existing is to be reachable from the outside, which means it's the host most exposed to the vulnerability that started this entire incident (CVE-2021-42013, a remotely exploitable flaw). MTB-FILE01 is an internal fleet file server, reached only via internal lateral movement, not directly from the internet. Bringing FILE01 back first lets you restore internal file services with a much smaller blast radius if something was missed, while WEB01 stays offline until you can be fully confident the internet-facing attack surface itself is closed. Reintroducing the DMZ host too early could re-open the exact door the attacker used.

6. State which backup generation is safe, based on your revised start of intrusion, and how you would verify it before restoring.

The revised start of intrusion is 2026-08-28 01:52:17 (first contact from 185.220.101.47, egress log). Any backup taken on or after August 28 is potentially within the compromise window and cannot be trusted as clean. The safe backup generation is the most recent one taken before August 28. Realistically, this likely means going back further than initially assumed, since even an August 27 backup would only be one day removed from first contact and worth extra scrutiny given the attacker's five-day dwell time before any exploitation activity began.

Verification before restoring from it: don't just trust the date stamp, actively check the backup itself against every indicator this investigation has produced. No svc-backup account in /etc/passwd, no attacker key in any authorized_keys file, no /etc/cron.d/apache-logrotate, no apache-worker/syslog-forward binaries, Apache still at the pre-attack version, and file hashes for anything restorable compared against the SHA-256 values already on record.

7. State what must be true before MTB-WEB01 is exposed to the internet again. Both vulnerabilities from your Day 1-2 assessment are part of this, and so is the Apache configuration.
 - CVE-2021-42013 patched. Apache upgraded to $\geq 2.4.51$, confirmed via `apache2 -v`, not just a package-manager claim.
 - CVE-2021-4034 (PwnKit) patched. policykit-1 upgraded to the security-fixed build, confirmed via the package changelog naming the CVE as resolved.
 - Apache configuration corrected. The `<Directory />` block set back to Require all denied with access explicitly scoped to /var/www/html, and ScriptAlias /cgi-bin/ either removed or tightly restricted, per the Day 1-2 finding (MTB-0001) that named this configuration as what made the version-level flaw actually exploitable. Patching the version without fixing this config leaves the host still restorable to the same failure mode by any other future vulnerability that touches the filesystem.
 - Every item in the Section 11.3 eradication table verified clean on this specific host (accounts, keys, cron entries, dropped files).
 - Host rebuilt or restored from the verified-clean backup generation, not patched in place on top of a potentially still-compromised filesystem. Patching known vulnerabilities doesn't remove unknown backdoors that might still be sitting there.

8. Define a heightened monitoring period: how long, watching for what, and using which of your four detection rules.

Duration: minimum 30 days from return to service

What to watch for:

- Any outbound connection to 185.220.101.47, 185.220.102.244, or any new address in the same address ranges
- Recurrence of svc-backup, or any new UID=0 account outside a documented break-glass change
- Any file written to /etc/cron.d/ or a new systemd unit executing a binary from /usr/local/
- Any script file appearing under /var/www/html/uploads/
- Repeat scanning/traversal patterns against /cgi-bin/

Which detection rules:

All four. mtb_detect_01 covers the cron/systemd persistence recurrence, mtb_detect_02 covers a repeat web shell drop, mtb_detect_03 covers the exact initial-access technique reoccurring, and mtb_detect_04 covers any lateral-movement attempt via a service account from within the server segment.

Root Cause and Contributing Factors

- **Why was a host with a publicly known, actively exploited vulnerability still exposed to the internet in September,** when the assessment on 8 September flagged it and it had been in the KEV catalogue since 2021? Your answer must address the patching process, not the attacker.

The KEV finding sat unpatched for years after public disclosure, this was purely a patch management failure.

- **Why did the assessment run on 8 September, after the first successful exploitation on 30 August?** What scan cadence would have caught this in time, and what would that cost?

The scan cadence was periodic, not continuous, so the gap between scans on this internet-facing host happened to contain the attack. Daily external scanning on DMZ assets would have closed that gap, at the cost of tooling and daily triage time for a small host count.

- **Why was an unattended SSH private key trusted by other hosts sitting on an internet-facing server in the DMZ?** What should the trust relationship between the DMZ and the server segment look like?

The DMZ and the internal server segment shared filesystem-level trust (a shared /home mount), so a key planted on the internet-facing host worked internally with no extra exploit needed. The DMZ should trust nothing from the internal segment, and nothing internal should trust anything from the DMZ.

- **Why did seven and a half hours of five-minute beaconing to a non-allowlisted destination not alert anyone sooner?** The egress log recorded every one of those connections. The data existed. What was missing?

The alert logic waited to confirm a repeating pattern before firing, even though the data existed from the first connection. What's missing is alerting on the first contact with a non-allowlisted destination, not waiting for periodicity to prove itself.

Recommendations

Recommendation	Owner	Due Date	Traceable to
Patch the web server software to the current secure version on MTB-WEB01	Infrastructure/Systems team	Within 24 hours	Root cause finding

Patch the privilege-escalation component (polkit) to the current secure version on MTB-WEB01 and MTB-FILE01	Infrastructure/Systems team	Within 24 hours	Privilege-escalation technique
Correct the web server's directory-access configuration to deny-by-default and remove or restrict the CGI script path	Infrastructure/Systems team	Within 24 hours, before MTB-WEB01 returns to service	Day 1–2 assessment configuration finding
Remove all attacker-created accounts, keys, scheduled tasks, and hidden programs from both MTB-WEB01 and MTB-FILE01	Incident Response team	Before either host returns to service	Eradication plan
Audit and, where needed, restructure the shared trust relationship between the internet-facing web server and internal servers so a credential planted on one cannot be used on another	Network/Infrastructure team	Within 5 business days	Lateral movement finding
Rotate all administrator and service-account credentials with access across more than one server, prioritizing accounts with reach into both the web server and the internal server segment	Identity/Access Management team	Within 48 hours	Containment plan follow-up
Move from periodic to continuous vulnerability scanning for all internet-facing systems, with a defined patching deadline for any finding already listed as actively exploited	Security/Vulnerability Management team	Within 10 business days	Scan-cadence finding
Change monitoring so a first-ever connection to a destination not on the approved list generates an immediate alert, rather than waiting to confirm a	SOC/Detection Engineering team	Within 5 business days	detection-gap finding

repeating pattern			
Deploy the four detection rules developed during this investigation, and the additional account-creation detection identified as a gap, across the full server fleet	SOC/Detection Engineering team	Within 3 business days	detection-gap review
Conduct a fleet-wide audit of every server for the specific indicators found in this incident, prioritizing the domain controller, which has not yet been fully cleared	Incident Response team	Within 48 hours	Intelligence gaps
Determine exactly what data was contained in the transfer from the file server, to close the current gap in understanding what was actually lost	Incident Response / Data Governance team	Within 5 business days	Data at risk / intelligence gaps
Maintain heightened monitoring on both affected servers for 30 days after they return to service, watching specifically for indicators from this incident	SOC team	Ongoing, starting immediately at service restoration	Recovery plan

13. Deliverables

Submit the following through the class shared drive.

13.1 Day 4

- Your extended timeline, merged with the Day 3 timeline, covering the full intrusion.
- Your completed fleet hunt grid, including the columns you added for newly discovered indicators.
- Your tiered indicator list, with a Pyramid tier and an attacker-cost note for every entry.
- Your three behaviour descriptions that name no address, filename or hash.
- Your incident scoping report, answering how far back, how many hosts, what data, with a stated dwell time and named intelligence gaps.

13.2 Day 5

- Your completed ATT&CK mapping, with evidence for every row and at least two techniques you added yourself.

- Four Sigma rule files, each with all mandatory fields plus description, tags, falsepositives and level.
- For each rule, the Pyramid tier it targets and what an attacker would change to evade it.
- Your containment, eradication and recovery plan, including the backup generation decision and the credential blast radius.
- Your post-incident review, including the detection gap table and answers to all four questions in Section 12.2.

14. Reflection Questions

Answer briefly in your own words, three to five sentences each. Graded on reasoning, not length.

14.1 Day 4

1. Your Day 3 forensic report gave a start date that turned out to be wrong by several days. The investigation was not careless. Explain what made the disk evidence misleading, and what that tells you about relying on any single evidence source.
2. The attacker used a different address, filename and persistence mechanism on the second host. What does that tell you about how they operate, and what would you now assume about a third host if you found one?
3. MTB-DVWA01 showed the same beacon-like periodicity that got MTB-WEB01 flagged, and it was benign. Explain what actually distinguished them, and why an analyst who cannot make that distinction is a problem for their team.
4. Why is documenting what you checked on the clean hosts as important as documenting what you found on the compromised ones?

14.2 Day 5

1. Explain the Pyramid of Pain in your own words, using an example from this incident, and say why blocking the C2 address after Day 3 achieved so little.
2. The account-creation rule in Section 10.3 detects the attacker using a warning the operating system produced by itself. Why is a detection built on the target's own logging harder to evade than one built on the attacker's tooling?
3. Every one of your rules will produce false positives eventually. Explain why the falsepositives field matters more to the analyst receiving the alert than to the person writing the rule.
4. The egress log recorded all seven and a half hours of beaconing. The data existed and nobody acted on it. What does that suggest about where security programmes actually fail, and how would you fix it here?

15. Grading Alignment

This lab counts toward your lab-completion and documentation-quality grading criteria.

For Day 4, the marks are in your scoping discipline. Anyone can find the compromised host once they are told to look. What is graded is whether a reader can tell what you checked, what you did not check, and how confident you are in each conclusion. A report that states certainty it has not earned will score below one that names its gaps.

For Day 5, the marks are in whether your detections would survive the attacker changing their infrastructure. Four rules that match addresses and hashes will score poorly however well formatted they are. Two good behavioural rules with honest false positive notes are worth more than four brittle ones.

Across both, your reports are graded on whether someone outside the incident could act on them. The executive summaries are for a person, not a scanner.

16. Completion Checklist

Work down this list before you submit. Every line is something that is graded. If you cannot tick a line, that is where to spend your remaining time.

16.1 Day 4

Done	Item
	Egress log copied into a spreadsheet and split into columns
	Revised start of intrusion identified, with the exact log row cited
	Difference between the revised start and the Day 3 start calculated in days
	Written explanation of why the disk image could not show the earlier activity
	All three external destination addresses identified
	Both large outbound transfers recorded, with host, timestamp and volume in megabytes
	Fleet hunt grid completed for all five hosts, every cell answered
	Second compromised host identified, with the SSH login timestamp and account
	Explanation of how the attacker authenticated to the second host
	All three tradecraft differences on the second host noted
	New indicator columns added to the grid and every host re-checked
	MTB-DVWA01 verdict reached, with the specific evidence that settles it
	For each clean host, the indicators and log sources checked are listed
	MTB-APP01 failed logins assessed and the judgement justified
	All sixteen or more indicators assigned a Pyramid tier
	Attacker cost to change recorded for every tier 1 and tier 2 indicator
	Three behaviour descriptions written that name no address, filename or hash
	Scoping report written with all seven sections from Section 8.2
	Dwell time calculated and the working shown
	Intelligence gaps named

16.2 Day 5

Done	Item
	All fifteen mapping rows completed with host, timestamp and source
	The two techniques implemented differently on each host identified
	The technique evidenced by decoding rather than observation identified

Done	Item
	At least two additional techniques added with evidence
	All fifteen tactics assessed for whether the attacker used them
	Four Sigma rule files written and saved separately
	Every rule has title, logsource, detection and condition
	Every rule has description, tags, falsepositives and level
	Every rule tagged with at least one ATT&CK technique in attack.tXXXX form
	Every falsepositives field names a specific realistic legitimate activity
	No rule matches an IP address, filename or hash as its only logic
	Pyramid tier and evasion note written for each rule
	Containment actions listed in order, with what each stops and what it risks
	Power-off versus isolate decision made and justified using order of volatility
	Containment actions for non-compromised hosts identified
	Eradication table completed with every access path on both hosts
	Credential and key blast radius stated, including every host trusting the SSH key
	Verification method for eradication stated
	Recovery order stated and justified
	Safe backup generation identified, based on the revised start of intrusion
	Conditions for re-exposing MTB-WEB01 to the internet stated
	Heightened monitoring period defined, with duration and which rules
	Detection gap table completed with all five timestamps
	Firing time and prevented impact stated for each of the four rules
	The single highest-value rule identified and justified
	What none of the rules would catch stated honestly
	All four questions in Section 12.2 answered, about process not people
	Post-incident review written with all eight sections from Section 12.3

Before you submit

Read both executive summaries out loud. If you stumble, or if you used a word a branch manager would not know, rewrite them.

Open each Sigma rule and ask the evasion question one more time: different address, different filenames, would it still fire?

Check that no live, clickable external address survives anywhere in your indicator list or your reports.

Appendix A: Reference Data and Troubleshooting

Everything in this appendix is here so that nothing can stop you. If a website is slow, blocked or down, the values you would have looked up are printed below. Use them, and note in your report that you used the printed reference rather than the live source.

A.1 MITRE ATT&CK reference

Current Enterprise tactics, in the order an intrusion typically moves through them. Note the names marked as changed - older material uses different words for the same thing.

ID	Tactic	Note
TA0043	Reconnaissance	
TA0042	Resource Development	
TA0001	Initial Access	
TA0002	Execution	
TA0003	Persistence	
TA0004	Privilege Escalation	
TA0005	Stealth	Renamed. Older material calls this Defense Evasion.
TA0006	Credential Access	
TA0007	Discovery	
TA0008	Lateral Movement	
TA0009	Collection	
TA0011	Command and Control	
TA0010	Exfiltration	
TA0040	Impact	
TA0112	Defense Impairment	Newer tactic. Not present in older material.

Techniques relevant to case MTB-IR-2026-0902. Identifiers and names as published by MITRE. Do the lookups yourself first - this table is a safety net and a way to check your work, not a shortcut past the exercise.

Technique ID	Official name	Tactic
T1595.002	Vulnerability Scanning	Reconnaissance
T1190	Exploit Public-Facing Application	Initial Access

Technique ID	Official name	Tactic
T1505.003	Web Shell	Persistence
T1068	Exploitation for Privilege Escalation	Privilege Escalation
T1136.001	Local Account	Persistence
T1098.004	SSH Authorized Keys	Persistence, Privilege Escalation
T1053.003	Cron	Execution, Persistence, Privilege Escalation
T1543.002	Systemd Service	Persistence, Privilege Escalation
T1078.003	Local Accounts	Stealth, Persistence, Privilege Escalation, Initial Access
T1552.004	Private Keys	Credential Access
T1021.004	SSH	Lateral Movement
T1046	Network Service Discovery	Discovery
T1033	System Owner/User Discovery	Discovery
T1083	File and Directory Discovery	Discovery
T1087.001	Local Account	Discovery
T1005	Data from Local System	Collection
T1560.001	Archive via Utility	Collection
T1071.001	Web Protocols	Command and Control
T1105	Ingress Tool Transfer	Command and Control
T1041	Exfiltration Over C2 Channel	Exfiltration
T1070.003	Clear Command History	Stealth
T1027	Obfuscated Files or Information	Stealth

A.2 Sigma reference

Allowed values for the fields that only accept a fixed set. Getting these wrong is the most common reason a rule fails validation.

Field	Allowed values
status	stable, test, experimental, deprecated, unsupported
level	informational, low, medium, high, critical
logsource	Any combination of category, product, service and definition
date and modified	ISO 8601 with separators only: YYYY-MM-DD
tags	Lower case, dot-separated namespaces, for example attack.t1053.003

A second complete rule, in a different shape from the one in Section 10.3, showing a named selection with a filter and a compound condition.

```

title: Outbound Session From Unmanaged Root Binary
id: 3d5b90e7-11ac-4c68-8b25-9f0e7c4a2d16
status: experimental
description: |
  Detects a long-lived outbound network session held by a root-owned process
  executing from a directory outside package manager control. Approved
  monitoring agents are excluded by the filter.
  Written following case MTB-IR-2026-0902.
references:
  - https://attack.mitre.org/techniques/T1071/001/
author: Meridian Trust Bank SOC
date: 2026-09-04
tags:
  - attack.t1071.001
  - attack.t1543.002
logsource:
  product: linux
  category: network_connection
detection:
  selection:
    User: 'root'
    Initiated: 'true'
    Image|startswith:
      - '/usr/local/bin/'
      - '/usr/local/sbin/'
  filter_approved:
    Image: '/opt/monagent/bin/monagent'
  condition: selection and not filter_approved
falsepositives:
  - Locally compiled administrative tooling installed outside the package
    manager. Should be inventoried; if it is not, that is its own finding.
level: high

```

A.3 CyberChef quick reference

Operations useful in this lab. Type the operation name into the search box at the top left of the Operations panel.

Operation	Category	Useful for
Extract IP addresses	Extractors	Pulling every address out of the egress log or a telemetry block
Sort	Utils	Ordering an extracted list
Unique	Utils	Collapsing a list of addresses down to the distinct ones
Count occurrences	Utils	Counting how many times one address appears across a log
Find / Replace	Utils	Reshaping a log block before pasting it into a spreadsheet
Regular expression	Regex	Extracting one field from every line, such as every dst_ip value
Defang IP Addresses	Networking	Making addresses safe before they go into a report
From Base64	Data format	Decoding an encoded payload string
SHA2	Hashing	Recomputing a file hash for verification

A.4 If something is not working

Problem	What to do
An ATT&CK sub-technique page returns not found	The identifier splits across the path. T1505.003 is at attack.mitre.org/techniques/T1505/003/ , not /T1505.003/ . This catches everyone once.
A technique name on the site does not match this document	MITRE revises names between releases. Use the name on the site today and note the difference. If a tactic says Defense Evasion anywhere, it now means Stealth.
attack.mitre.org is slow or will not load	Appendix A.1 prints every technique used in this incident with its official name and tactic. Say in your report that you used the printed reference.
My Sigma rule looks wrong and I cannot see why	Nine times out of ten it is indentation. Check for tab characters, and check that every level is indented by four spaces. A colon inside a value such as a file path must be inside quotes.
I cannot decide what logsource to use	There is no single correct answer, and that is fine. Pick the combination that best describes where the evidence came from, and state your assumption in the description field. Being explicit about the assumption is what is graded.
I cannot think of a false positive for my rule	That usually means the rule is too broad, or you have not asked who else does this legitimately. Backup software, configuration management and monitoring agents do things that look like attacker behaviour. Start there.
The hunt grid gives me a host I am not sure about	That is the intended experience for one of the five hosts. Section 6.4 holds the evidence that settles it. Read the asset record before you decide.
I cannot find the second large data transfer	Sort the egress log by the bytes_out column, largest first. The two rows you want are thousands of times larger than everything else and come from different hosts.

A.5 Working with an AI assistant

You may use an AI assistant on this lab, and you should. Use it the way a working analyst does.

- **Good use** - ask it to explain what a systemd unit does, what YAML indentation rules are, why a particular field exists in Sigma, or to review whether your executive summary reads clearly to a non-technical person.
- **Bad use** - asking it to write your four detection rules. You will get four rules that look plausible and you will not be able to defend a single design decision in them, which is the whole assessment.
- **Always verify** - AI assistants are frequently wrong about ATT&CK technique identifiers and about which tactic a technique belongs to. They are also often working from older ATT&CK releases, so they will tell you a technique sits under Defense Evasion. Check every identifier against attack.mitre.org or Appendix A.1 before it goes in your report.

Cite it. If an AI helped you reach a conclusion, note where and what you did to verify it. That is the standard a regulated bank holds analysts to, and it is the standard here.